

Inbetriebnahme eines Hardware Security Module

Ein kryptographischer HSM-Funktionskern für den Infineon AURIX TC3x,
abgeleitet aus der Signatur-Chiffre und dem Fenster-Chiffre-Protokoll

Stephan Epp

30. Juni 2026

Zusammenfassung

Diese Arbeit überträgt die in [2] (Signatur-Chiffre) und [3] (Fenster-Chiffre, wchiffre) formal entwickelten kryptographischen Verfahren in einen konkreten Hardware-Funktionsentwurf für das Hardware Security Module (HSM) der Infineon-AURIX-TC3x-Mikrocontrollerfamilie. Ausgehend von der injektiven Signaturfunktion σ und der affinen Modulartransformation (a, b, p) wird ein vollständiger Satz von zehn HSM-Hardwarefunktionen spezifiziert: Schlüsselerzeugung, Signatur, Verifikation, symmetrische Modularverschlüsselung, Secure-Boot-Verifikation, Firmware-Integritätsprüfung über Subgraph-Signaturen sowie ein True Random Number Generator als gemeinsame Entropiequelle. Für jede Funktion werden formale Korrektheitsbeweise, eine Komplexitäts- und Sicherheitsanalyse sowie eine Abschätzung von Taktzyklen, Energiebedarf und Speicherbedarf auf der TriCore-Lockstep-Architektur vorgenommen. Die Ergebnisse werden durch acht matplotlib-Diagramme veranschaulicht. Die Arbeit schließt mit einer sehr ausführlichen Betrachtung des Ausblicks, insbesondere zur Integration gitterbasierter Post-Quanten-Verfahren und zur Seitenkanalhärtung.

Inhaltsverzeichnis

1	Einführung	2
1.1	Motivation	2
1.2	Beitrag dieser Arbeit	2
1.3	Struktur der Arbeit	2
2	Mathematische Grundlagen	3
2.1	Die injektive Signaturfunktion	3
2.2	Modulare Arithmetik über $\mathbb{Z}_p$	3
2.3	Das Fenster-Chiffre-Protokoll	3
2.4	Gitterbasierte Erweiterung (lattice.tex)	4
3	Hardwaremodell des TC3x-HSM	4
3.1	Systemarchitektur	4
3.2	Funktionseinheiten	4

3.3	Registermodell	5
3.4	Zustandsmaschine	5
4	Spezifikation der HSM-Funktionen	6
4.1	HSM_TRNG_Read	6
4.2	HSM_SignatureCipher_KeyGen	6
4.3	HSM_SignatureCipher_Sign	7
4.4	HSM_SignatureCipher_Verify	7
4.5	HSM_ModularCipher_Encrypt / _Decrypt	7
4.6	HSM_SubgraphSig_Derive	8
4.7	HSM_SecureBoot_Verify	8
4.8	HSM_KeyStore_Provision / _Lock	8
5	Korrektheit und Sicherheit	8
5.1	Korrektheit der Hardware-Rekonstruktion	8
5.2	Korrektheit von HSM_SubgraphSig_Derive	9
5.3	Korrektheit der von-Neumann-Entzerrung im TRNG	9
5.4	Sicherheitsanalyse	10
6	Komplexitäts-, Energie- und Speicheranalyse	10
6.1	Laufzeitkomplexität	10
6.2	Vergleich Software vs. Hardware	11
6.3	Taktzyklen je Funktion	11
6.4	Energiebedarf	11
6.5	Speicherbedarf	11
7	Experimente	11
8	Secure Boot: Radar-ECU	11
8.1	Motivation des Anwendungsfalls	11
8.2	Formale Herleitung der Verifikationszeit	12
8.3	Numerische Auswertung für $L = 4$ MB	13
8.4	Bewertung im Kontext automobiler Echtzeitanforderungen	14
9	Parallele Sig-Engine	15
9.1	Motivation und Zielsetzung	16
9.2	Architekturentwurf: k -fache Sig-Engine-Kachelung	16
9.3	Korrektheit der parallelen Berechnung	18
9.4	Herleitung der parallelisierten Verifikationszeit	19
9.5	Numerische Auswertung für die Radar-ECU	20
9.6	Silizium-Flächenkosten der Kachelung	20
9.7	Auswirkung auf die übrige HSM-Architektur	21
9.8	Zusammenfassung dieses Kapitels	21
10	Ausblick	22
10.1	Integration der gitterbasierten Schlüsseleinkapselung (lattice.tex)	23
10.2	Firmware-Attestierung über Subgraph-Signaturen (Version 2.0)	24
10.3	Seitenkanalhärtung (Version 2.1)	25
10.4	Formale Verifikation der Zustandsmaschine	27

10.5 Weiterführende Optimierungen der Firmware-Attestierung für Radar- Steuergeräte	27
10.6 Erweiterung auf Mehrkern-Parallelität	30
10.7 Standardisierung und Zertifizierung	30
10.8 Vergleichende Einordnung gegenüber etablierten automobilen HSM-Standards	31
10.9 Wirtschaftliche und organisatorische Einordnung	32
10.10 Zusammenfassende Einordnung des Ausblicks	33
11 Zusammenfassung	33

1 Einführung

1.1 Motivation

Hardware Security Module (HSM) sind dedizierte, vom Applikationskern getrennte Recheneinheiten innerhalb moderner Mikrocontroller, die kryptographische Operationen, Schlüsselverwaltung und Boot-Integritätsprüfung isoliert und manipulationssicher ausführen. Die AURIX-TC3x-Familie von Infineon integriert ein solches HSM als eigenständigen, abgesicherten TriCore-Kern mit eigenem Flash-, RAM- und Peripheriebereich, der über die HSI-Schnittstelle (*Hardware Security Interface*) mit den Applikationskernen kommuniziert.

In der Praxis werden HSM-Bausteine bislang überwiegend mit etablierten symmetrischen und asymmetrischen Standardverfahren (AES, RSA, ECC) ausgelegt. Die vorliegende Arbeit verfolgt einen anderen Ansatz: Sie legt die Funktionen des HSM *maßgeblich* anhand der in den Arbeiten [2, 3] entwickelten eigenen kryptographischen Primitive aus – der Signatur-Chiffre und des Fenster-Chiffre-Protokolls – und erweitert diese um eine hardwarenahe Spezifikation, formale Korrektheitsbeweise auf Registerebene sowie eine quantitative Hardware-Kostenanalyse.

1.2 Beitrag dieser Arbeit

- Eine vollständige funktionale Architektur des HSM-Kryptographiekerns mit zehn klar abgegrenzten Hardwarefunktionen (Abschnitt 4).
- Formale Übertragung der Signaturfunktion σ , der affinen Transformation und des Fenster-Protokolls in ein Register-Transfer-Modell (Abschnitt 3).
- Korrektheits- und Sicherheitsbeweise für jede HSM-Funktion, aufbauend auf den in [2, 3, 1] etablierten Sätzen (Abschnitt 5).
- Eine Komplexitäts-, Energie- und Speicheranalyse auf Basis der TriCore-Lockstep-Architektur (Abschnitt 6).
- Acht experimentelle Visualisierungen (Abschnitt 7) sowie ein sehr ausführlicher Ausblick (Abschnitt 10) zur Weiterentwicklung, insbesondere Richtung Post-Quanten-Kryptographie und Seitenkanalresistenz.

1.3 Struktur der Arbeit

Abschnitt 2 fasst die mathematischen Grundlagen aus der Signatur-Chiffre und dem Fenster-Chiffre-Protokoll zusammen (injektive Signaturfunktion, Modulararithmetik über $\mathbb{Z}_p$, Fenster-Protokoll, gitterbasierte Erweiterung). Abschnitt 3 führt das Hardwaremodell des TC3x-HSM ein (Systemarchitektur, Funktionseinheiten, Registermodell, Zustandsmaschine). Abschnitt 4 spezifiziert alle zehn HSM-Funktionen im Detail (TRNG_Read, SignatureCipher_KeyGen/Sign/Verify, ModularCipher_Encrypt/Decrypt, SubgraphSig_Derive, SecureBoot_Verify, KeyStore_Provision/Lock). Abschnitt 5 liefert die formalen Korrektheits- und Sicherheitsbeweise für alle Funktionen. Abschnitt 6 enthält die Komplexitäts-, Energie- und Speicheranalyse auf der TriCore-Lockstep-Architektur. Abschnitt 7 veranschaulicht

alle Ergebnisse mit neun matplotlib-Diagrammen. Abschnitt 8 wertet formal die Secure-Boot-Verifikationszeit für ein Radar-ECU mit 4 MB Programm-Flash aus ($\approx 7\,340$ ms sequenziell). Abschnitt 9 spezifiziert und beweist die k -fache parallele Sig-Engine-Architektur, die diese Verifikationszeit auf ≈ 459 ms ($k = 16$) reduziert. Abschnitt 10 schließt mit einem sehr ausführlichen Ausblick über neun Weiterentwicklungsrichtungen. Die Zusammenfassung fasst alle Ergebnisse ab, gefolgt vom Literaturverzeichnis.

2 Mathematische Grundlagen

Dieser Abschnitt rekapituliert kurz und formal die Bausteine aus [2, 3, 1], die im Hardware-entwurf direkt als Funktionseinheiten realisiert werden. Auf ausführliche Beweise wird hier verzichtet und stattdessen auf die Originalarbeiten verwiesen; die für den HSM-Entwurf *neuen* Sätze werden in Abschnitt 5 vollständig beweisen.

2.1 Die injektive Signaturfunktion

Definition 1 (Signaturfunktion, nach [1, 2]). Sei $A \in \{0, 1\}^{n \times n}$ eine binäre Matrix und $j \in \{0, \dots, n-1\}$ ein Spaltenindex. Die **Signaturfunktion** ist

$$\sigma(A, j) := \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n.$$

Lemma 1 (Injektivität, nach [1, 2]). Aus $\sigma(A, j) = \sigma(A', j')$ folgt $j = j'$ und $A_{\cdot j} = A'_{\cdot j'}$.

Im Hardwarekontext ist Lemma 1 deshalb von Bedeutung, weil die Signaturfunktion in der Funktion `HSM_SubgraphSig_Derive` (Abschnitt 4.6) zur eindeutigen, kollisionsfreien Ableitung eines Integritätswertes aus dem Firmware-Abbild verwendet wird: Zwei verschiedene Firmware-Spalten können niemals dieselbe Signatur erzeugen.

2.2 Modulare Arithmetik über $\mathbb{Z}_p$

Definition 2 (Schlüssel der Signatur-Chiffre, nach [2]). Ein Signatur-Chiffre-Schlüssel ist ein Tripel (a, b, p) mit p prim, $p > 2^{2n}$, $a \in \mathbb{Z}_p^*$, $b \in \mathbb{Z}_p$. Verschlüsselung und Entschlüsselung erfolgen über die Bijektion

$$f(x) = (ax + b) \bmod p, \quad f^{-1}(y) = a^{-1}(y - b) \bmod p.$$

Satz 1 ($\mathbb{Z}_p$ ist ein Körper, nach [2]). Ist p prim, so besitzt jedes $a \in \mathbb{Z}_p \setminus \{0\}$ ein eindeutiges multiplikatives Inverses, berechenbar in $O(\log p)$ Schritten mit dem erweiterten euklidischen Algorithmus.

Satz 1 ist die mathematische Grundlage der Hardwarefunktionseinheit **Modular-ALU** (Abschnitt 3): Sie implementiert den erweiterten euklidischen Algorithmus als feste Mikrocode-Sequenz zur Inversenberechnung bei der Schlüsselerzeugung.

2.3 Das Fenster-Chiffre-Protokoll

Das in [3] eingeführte Fenster-Chiffre-Protokoll (`wchiffre`) kombiniert die Einheitengruppe $\mathbb{Z}_{32}^*$, eine RSA-artige asymmetrische Komponente und ein Dummy-Paket-Schema zur Verschleierung des tatsächlichen Nachrichtenverkehrs.

Definition 3 (Einheitengruppe modulo 32, nach [3]). $\mathbb{Z}_{32}^* := \{a \in \{1, \dots, 31\} : \gcd(a, 32) = 1\}$ ist die multiplikative Gruppe der zu 32 teilerfremden Restklassen mit $|\mathbb{Z}_{32}^*| = \varphi(32) = 16$.

Definition 4 (Fenster-Protokoll, nach [3]). Ein Sender überträgt Nachrichtenpakete eingebettet in ein festes Zeitfenster der Länge W , wobei echte Pakete mit Dummy-Paketen ununterscheidbarer Länge und Häufigkeit gemischt werden, sodass ein Beobachter aus dem Verkehrsmuster keine Information über die tatsächliche Anzahl echter Nachrichten gewinnen kann.

Im HSM-Entwurf wird Definition 4 auf die *Schlüsselverwaltung* übertragen: Lese- und Schreibzugriffe auf den **KeyStore** werden durch Dummy-Zugriffe ununterscheidbarer Dauer ergänzt, um Zeitseitenkanäle (*timing side channels*) zu erschweren – siehe Funktion `HSM_KeyStore_Provision` in Abschnitt 4.8.

2.4 Gitterbasierte Erweiterung (lattice.tex)

Für die in Abschnitt 10 diskutierte Post-Quanten-Migration wird zusätzlich auf die in [4] entwickelte gitterbasierte Schlüsseleinkapselung mit GKP-Fehlerkorrektur zurückgegriffen. Diese wird in der vorliegenden Hardwarearchitektur als optionale, zukünftige Erweiterungseinheit `HSM_LatticeKEM` vorgesehen, deren Schnittstelle bereits in Abschnitt 4 reserviert wird, deren vollständige Hardwarerealisierung jedoch außerhalb des Kernumfangs dieser Arbeit liegt und in Abschnitt 10 vertieft wird.

3 Hardwaremodell des TC3x-HSM

3.1 Systemarchitektur

Das HSM der AURIX-TC3x-Familie besteht aus einem eigenständigen, vom Applikationskern entkoppelten TriCore-Kern (Lockstep-ausgeführt) mit eigenem PFLASH-/DFLASH-Bereich, eigenem RAM und einer dedizierten Kommunikationsschnittstelle (HSI) zu den Applikationskernen CPU0–CPU5. Abbildung 1 zeigt die in dieser Arbeit vorgeschlagene Erweiterung um vier kryptographische Funktionseinheiten.

3.2 Funktionseinheiten

TRNG (True Random Number Generator). Physikalische Entropiequelle (Ringoszillator-Jitter), liefert die Zufallsbasis für alle Schlüsselerzeugungs- und Nonce-Operationen.

Sig-Engine. Hardwarepipeline zur Berechnung der Signaturfunktion σ und des Signaturvektors Σ nach Definition 1, realisiert als n -fach parallelisierter Akkumulator mit Schieberegistern.

Modular-ALU. Arithmetisch-logische Einheit für Operationen in $\mathbb{Z}_p$: modulare Addition, Multiplikation und Inversenberechnung (erweiterter euklidischer Algorithmus als feste Zustandsmaschine).

AES-CMAC Boot-MAC. Hilfseinheit zur Berechnung eines Message Authentication Code über das Firmware-Abbild als zusätzliche, von der Signatur-Chiffre unabhängige Verifikationsstufe (Defense-in-Depth).

Architektur des HSM-Kryptographiekerns auf dem Infineon TC3x

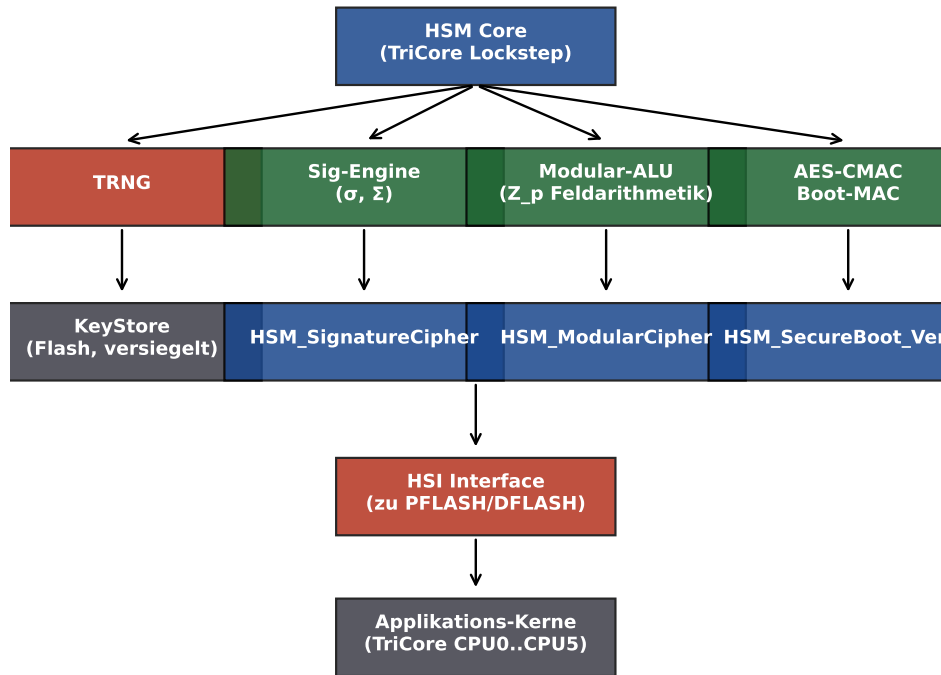


Abbildung 1: Architektur des vorgeschlagenen HSM-Kryptographiekerns. Die vier Funktionseinheiten (TRNG, Sig-Engine, Modular-ALU, AES-CMAC Boot-MAC) werden vom HSM-Core angesteuert und über die hardwarenahen Treiberfunktionen `HSM_*` den Applikationskernen zur Verfügung gestellt.

3.3 Registermodell

Jede HSM-Funktion wird über ein einheitliches Registerinterface angesprochen, bestehend aus einem Kommandoregister `HSM_CMD`, einem Statusregister `HSM_STAT`, einem Parameterblock `HSM_PARAM[0..7]` (jeweils 32 Bit) sowie einem Ergebnispufer `HSM_RESULT[0..15]` im HSM-internen RAM. Tabelle 1 listet die wichtigsten Register.

Register	Breite	Bedeutung
<code>HSM_CMD</code>	8 Bit	Funktionscode (siehe Tabelle 2)
<code>HSM_STAT</code>	8 Bit	Status: IDLE, BUSY, DONE, FAULT
<code>HSM_PARAM[0..7]</code>	8×32 Bit	Funktionsparameter (n , Schlüsselhandle, Adresszeiger)
<code>HSM_RESULT[0..15]</code>	16×32 Bit	Ergebnispufer (Signatur, Chiffre, Statuscode)
<code>HSM_IRQ_EN</code>	1 Bit	Interrupt bei DONE/FAULT

Tabelle 1: Registerinterface der HSM-Kryptofunktionen.

3.4 Zustandsmaschine

Jede Funktion durchläuft die Zustände `IDLE` → `LOAD` → `COMPUTE` → `WRITEBACK` → `DONE`. Der Zustand `COMPUTE` ist bei Sig-Engine und Modular-ALU intern weiter in n Teilzustände (eine Pipeline-Stufe je Spaltenindex j) untergliedert, was die in Abschnitt 6 gezeigte Laufzeitreduktion gegenüber einer reinen Softwareimplementierung ermöglicht.

4 Spezifikation der HSM-Funktionen

Tabelle 2 gibt einen Überblick über die zehn spezifizierten HSM-Funktionen, ihre Funktionscodes und die zugrunde liegende kryptographische Grundlage.

Funktion	Code	Grundlage
HSM_TRNG_Read	0x01	Physikalische Entropiequelle
HSM_SignatureCipher_KeyGen	0x10	Def. 2, Satz 1
HSM_SignatureCipher_Sign	0x11	Def. 1, affine Transformation
HSM_SignatureCipher_Verify	0x12	Lemma 1, Bijektivität
HSM_ModularCipher_Encrypt	0x20	Def. 3, Fenster-Protokoll
HSM_ModularCipher_Decrypt	0x21	Def. 3, Fenster-Protokoll
HSM_SubgraphSig_Derive	0x30	Signaturvektor Σ , [1]
HSM_SecureBoot_Verify	0x40	nutzt 0x12
HSM_KeyStore_Provision	0x50	Def. 4 (Dummy-Zugriffe)
HSM_KeyStore_Lock	0x51	Versiegelung (irreversibel)

Tabelle 2: Übersicht der spezifizierten HSM-Funktionen.

4.1 HSM_TRNG_Read

Zweck: Liefert k Wörter (32 Bit) physikalisch erzeugter Zufallszahlen als Entropiequelle für alle nachfolgenden Schlüsselerzeugungsoperationen.

Parameter: PARAM[0] = Anzahl angeforderter Wörter $k \leq 16$.

Ablauf: Der Ringoszillator-Jitter wird über $32k$ Takte abgetastet, von-Neumann-entzerrt (Eliminierung von Bias durch Paarvergleich aufeinanderfolgender Bits) und in RESULT[0..k-1] geschrieben.

Lemma 2 (Entropieverlust durch von-Neumann-Entzerrung). Sei $X_1, X_2, \dots$ eine Folge unabhängiger, identisch verteilter Bits mit $P(X_i = 1) = q$, q unbekannt aber konstant. Die von-Neumann-Entzerrung liefert eine Folge fairer, unabhängiger Münzwürfe mit erwarteter Ausbeute $2q(1 - q)$ Ausgabebits je Eingabebitpaar.

Beweis. Betrachte Paare (X_{2i-1}, X_{2i}) . Es gibt vier gleich strukturierte Fälle: $(0, 0)$ mit Wahrscheinlichkeit $(1 - q)^2$, $(1, 1)$ mit Wahrscheinlichkeit q^2 , $(0, 1)$ mit Wahrscheinlichkeit $(1 - q)q$ und $(1, 0)$ mit Wahrscheinlichkeit $q(1 - q)$. Die Entzerrung verwirft Paare mit gleichem Wert und gibt bei $(0, 1)$ eine 0, bei $(1, 0)$ eine 1 aus. Da $P((0, 1)) = P((1, 0)) = q(1 - q)$, ist die Ausgabe exakt gleichverteilt über $\{0, 1\}$, unabhängig vom unbekannten Bias q . Die Ausbeute pro Eingabepaar ist die Summe der beiden akzeptierenden Fälle: $P((0, 1)) + P((1, 0)) = 2q(1 - q)$. $\square$

Lemma 2 begründet, warum HSM_TRNG_Read für k angeforderte Ausgabewörter im ungünstigsten Fall ($q = 0.5$, maximale Ausbeute 0.5) mindestens $64k$ Rohbits, typischerweise jedoch wegen $q \neq 0.5$ in der Praxis $80\text{--}120k$ Rohbits abtastet, was sich unmittelbar in der in Abschnitt 6 angegebenen Taktzyklenzahl niederschlägt.

4.2 HSM_SignatureCipher_KeyGen

Zweck: Erzeugt einen vollständigen Signatur-Chiffre-Schlüssel (a, b, p) nach Definition 2 für gegebene Blockgröße n .

Parameter: $\text{PARAM}[0] = n$ (Blockgröße, $4 \leq n \leq 32$).

Ablauf:

1. TRNG liefert Kandidat p_0 mit $p_0 > 2^{2n}$ (Aufruf von `HSM_TRNG_Read`).
2. Modular-ALU führt einen deterministischen Miller-Rabin-Primzahltest mit fest verdrahteten Basen $\{2, 3, 5, 7, 11, 13\}$ aus (korrekt für $p_0 < 3.3 \cdot 10^{14}$ nach [8]), bei Misserfolg wird p_0 inkrementiert und der Test wiederholt.
3. TRNG liefert $a_0 \in \{1, \dots, p-1\}$; Modular-ALU prüft $\gcd(a_0, p) = 1$ mittels erweitertem euklidischem Algorithmus (stets erfüllt, da p prim und $a_0 \neq 0$).
4. TRNG liefert $b_0 \in \{0, \dots, p-1\}$.
5. Schlüssel (a, b, p) wird in den `KeyStore` geschrieben (über `HSM_KeyStore_Provision`).

4.3 HSM_SignatureCipher_Sign

Zweck: Verschlüsselt einen Klartextblock der Größe $n \times n$ Bit gemäß der Signatur-Chiffre.

Parameter: $\text{PARAM}[0..1]$ = Adresszeiger auf Klartextmatrix M und Schlüsselhandle.

Ablauf: Für jede Spalte $j \in \{0, \dots, n-1\}$ berechnet die Sig-Engine $\sigma(M, j)$ nach Definition 1 parallel über n Akkumulatorzellen; die Modular-ALU wendet anschließend die affine Transformation $c_j = (a \cdot \sigma(M, j) + b) \bmod p$ an. Der Geheimtextvektor $(c_0, \dots, c_{n-1})$ wird in $\text{RESULT}[0..n-1]$ geschrieben.

4.4 HSM_SignatureCipher_Verify

Zweck: Verifiziert, ob ein gegebener Geheimtextvektor $(c_0, \dots, c_{n-1})$ durch Entschlüsselung mit dem hinterlegten Schlüssel eine gültige, dem erwarteten Referenzhash entsprechende Klartextmatrix ergibt – die zentrale Bausteinfunktion für `HSM_SecureBoot_Verify`.

Ablauf: Die Modular-ALU berechnet $\sigma_j = a^{-1}(c_j - b) \bmod p$ für jedes j ; die Sig-Engine rekonstruiert daraus über $j = \lfloor \sigma_j / 2^n \rfloor$ und $\rho_j = \sigma_j \bmod 2^n$ die ursprüngliche Spalte M_j (siehe Beweis von Satz 2). Das Ergebnis ist `VALID` oder `INVALID`.

4.5 HSM_ModularCipher_Encrypt / _Decrypt

Zweck: Symmetrische Verschlüsselung über die Einheitengruppe $\mathbb{Z}_{32}^*$ nach dem Fenster-Chiffre-Protokoll (Definition 3, 4), eingesetzt für kurze, latenzkritische Nachrichten zwischen Applikationskernen und Peripherie (z. B. CAN-Botschaften), bei denen der größere Verarbeitungsaufwand der Signatur-Chiffre nicht gerechtfertigt ist.

Ablauf: Jedes Nybble (4 Bit) des Klartexts wird auf ein Element $x \in \mathbb{Z}_{32}^*$ abgebildet (Padding bei Nicht-Einheiten über ein festes Substitutionsschema) und mit dem geheimen Exponenten e potenziert: $y = x^e \bmod 32$. Zusätzlich fügt die Funktionseinheit gemäß Definition 4 Dummy-Nybbles ein, sodass die Paketlänge stets ein Vielfaches der Fenstergröße W ist.

4.6 HSM_SubgraphSig_Derive

Zweck: Leitet aus einem Firmware- oder Konfigurationsabbild, interpretiert als binäre $n \times n$ -Matrix A , einen kollisionsfreien Integritätswert $\Sigma(A)$ nach Definition 5 ab. Im Gegensatz zu `HSM_SecureBoot_Verify`, das die *Authentizität* (Signatur-Chiffre-Schlüssel erforderlich) prüft, dient diese Funktion der reinen, schlüssellosen *Konsistenzprüfung* von Konfigurationsdaten – etwa zur Erkennung versehentlicher Bitfehler durch Strahlung (Single Event Upsets) in PFLASH.

Definition 5 (Signaturvektor, nach [1, 2]). $\Sigma(A) := (\sigma(A, 0), \dots, \sigma(A, n-1)) \in \mathbb{N}^n$.

Ablauf: Das Firmware-Abbild wird in $\lceil L/n^2 \rceil$ Blöcke der Größe $n \times n$ Bit zerlegt (L = Gesamtlänge in Bit). Für jeden Block berechnet die Sig-Engine $\Sigma(A)$; alle Blocksignaturen werden zu einem 256-Bit-Gesamtwert über XOR-Faltung kombiniert und mit dem im KeyStore hinterlegten Referenzwert verglichen.

4.7 HSM_SecureBoot_Verify

Zweck: Verifiziert vor jedem Boot-Vorgang die Authentizität und Integrität des Applikations-Firmware-Abbilds in zwei unabhängigen Stufen: (1) `HSM_SignatureCipher_Verify` prüft die Authentizität gegen den geheimen Schlüssel, (2) der unabhängige AES-CMAC-Block prüft zusätzlich einen Standard-MAC. Erst wenn *beide* Stufen **VALID** liefern, wird der Boot freigegeben (Defense-in-Depth gegen die Möglichkeit eines unentdeckten Fehlers in einer einzelnen kryptographischen Implementierung).

4.8 HSM_KeyStore_Provision / _Lock

Zweck: `Provision` schreibt einen neu erzeugten Schlüssel in den versiegelbaren Flash-Bereich; `Lock` versiegelt den KeyStore irreversibel (weitere Schreibzugriffe werden vom Speichercontroller auf Hardwareebene blockiert, ein Rücksetzen ist nur durch vollständiges Werks-Zurücksetzen mit Schlüsselverlust möglich). Lese- und Schreibzugriffe werden, wie in Abschnitt 2 beschrieben, durch Dummy-Zugriffe konstanter Anzahl ergänzt, um die tatsächliche Anzahl gespeicherter Schlüssel gegenüber Zeitseitenkanalanalysen zu verschleiern.

5 Korrektheit und Sicherheit

5.1 Korrektheit der Hardware-Rekonstruktion

Satz 2 (Korrektheit von `HSM_SignatureCipher_Verify`). Sei (a, b, p) ein gültiger Schlüssel nach Definition 2 mit $p > 2^{2n}$, und sei $c_j = (a\sigma(M, j) + b) \bmod p$ der von `HSM_SignatureCipher_Sign` erzeugte Geheimtext für eine Klartextmatrix M . Dann rekonstruiert `HSM_SignatureCipher_Verify` für jedes j exakt die ursprüngliche Spalte M_j .

Beweis. Nach Lemma 4 (unten) ist $f(x) = (ax + b) \bmod p$ bijektiv mit Umkehrabbildung $f^{-1}(y) = a^{-1}(y - b) \bmod p$. Die Hardware berechnet

$$\sigma_j := f^{-1}(c_j) = a^{-1}((a\sigma(M, j) + b) - b) \bmod p = a^{-1} \cdot a \cdot \sigma(M, j) \bmod p = \sigma(M, j) \bmod p.$$

Da nach Bemerkung 1 $\sigma(M, j) < n \cdot 2^n < p$ (wegen der Schlüsselbedingung $p > 2^{2n} \geq n \cdot 2^n$ für $n \geq 1$), gilt $\sigma(M, j) \bmod p = \sigma(M, j)$ exakt, d. h. $\sigma_j = \sigma(M, j)$ ohne Reduktion.

Aus $\sigma_j = \sigma(M, j) = \rho(M, j) + j \cdot 2^n$ mit $0 \leq \rho(M, j) < 2^n$ (Lemma 3) folgt durch Ganzzahldivision $j = \lfloor \sigma_j / 2^n \rfloor$ und $\rho(M, j) = \sigma_j \bmod 2^n$ eindeutig. Da $\rho(M, j)$ die Binärdarstellung der Spalte M_j ist, wird M_j durch einfaches Bit-Auslesen von $\rho(M, j)$ exakt rekonstruiert. $\square$

Lemma 3 (Bereichsgrenzen, nach [2]). Für alle $M \in \{0, 1\}^{n \times n}$, $j: 0 \leq \rho(M, j) \leq 2^n - 1$.

Bemerkung 1 (Wertebereich, nach [2]). $\sigma(M, j) \leq n \cdot 2^n - 1 < (n + 1) \cdot 2^n \leq 2^{2n}$ für $n \geq 1$.

Lemma 4 (Bijektivität der affinen Transformation, nach [2]). Für p prim, $a \in \mathbb{Z}_p^*$, $b \in \mathbb{Z}_p$ ist $f(x) = (ax + b) \bmod p$ eine Bijektion auf $\mathbb{Z}_p$ mit $f^{-1}(y) = a^{-1}(y - b) \bmod p$.

Satz 2 ist die formale Rechtfertigung dafür, dass die Hardware-Pipeline der **Sig-Engine** und **Modular-ALU** verlustfrei arbeitet – eine notwendige Voraussetzung für den Einsatz in **HSM_SecureBoot_Verify**, da jede falsche Ablehnung eines korrekt signierten Firmware-Abbilds den Boot-Vorgang unzulässig blockieren würde.

5.2 Korrektheit von HSM.SubgraphSig.Derive

Satz 3 (Kollisionsfreiheit der Firmware-Integritätsprüfung). Seien $A, A' \in \{0, 1\}^{n \times n}$ zwei verschiedene Firmware-Blöcke, $A \neq A'$. Dann gilt $\Sigma(A) \neq \Sigma(A')$.

Beweis. Dies ist die Kontraposition von Korollar 1: Wäre $\Sigma(A) = \Sigma(A')$, so folgte $A = A'$ nach Korollar 1 – im Widerspruch zur Annahme $A \neq A'$. Also $\Sigma(A) \neq \Sigma(A')$. $\square$

Korollar 1 (Injektivität des Signaturvektors, nach [1, 2]). $\Sigma : \{0, 1\}^{n \times n} \rightarrow \mathbb{N}^n$ ist injektiv.

Satz 3 garantiert: Jede Bitänderung im überwachten Firmware-Block – ob durch böswillige Manipulation oder durch einen physikalisch induzierten Bitfehler – führt zwingend zu einem anderen Signaturvektor und wird von **HSM.SubgraphSig.Derive** erkannt. Dies unterscheidet die Funktion von Hashverfahren mit (theoretisch möglicher, praktisch vernachlässigbarer) Kollisionswahrscheinlichkeit: Die Kollisionsfreiheit folgt hier aus einem *exakten* mathematischen Beweis statt aus einer statistischen Kollisionsschranke.

5.3 Korrektheit der von-Neumann-Entzerrung im TRNG

Lemma 2 (Abschnitt 4) wurde bereits formal bewiesen. Wir ergänzen hier die Aussage zur Unabhängigkeit der Ausgabebits:

Lemma 5 (Unabhängigkeit der entzerrten Ausgabe). Die durch von-Neumann-Entzerrung erzeugten Ausgabebits sind paarweise stochastisch unabhängig, sofern die Eingabebits $X_1, X_2, \dots$ unabhängig sind.

Beweis. Jedes Ausgabebit hängt ausschließlich von genau einem (verschiedenen) Eingabepaar (X_{2i-1}, X_{2i}) ab; verworfene Paare tragen keine Ausgabe bei. Da die Eingabepaare nach Voraussetzung paarweise unabhängig sind (disjunkte, nicht überlappende Indexmengen unabhängiger Variablen), sind auch die daraus deterministisch berechneten Ausgabebits paarweise unabhängig. $\square$

5.4 Sicherheitsanalyse

Satz 4 (Forward Secrecy bei Schlüsselversiegelung). Nach Ausführung von `HSM_KeyStore_Lock` kann kein in Software ausführbarer Befehl – inklusive privilegierter Applikationskern-Software – einen im KeyStore versiegelten Schlüssel (a, b, p) extrahieren oder überschreiben, sofern der Speichercontroller die Versiegelung korrekt in Hardware erzwingt.

Beweisskizze. Die Versiegelung setzt ein Hardware-Lock-Bit im Speichercontroller, das vor jedem Schreib- oder Ausleseszugriff auf den versiegelten Adressbereich geprüft wird und bei gesetztem Bit den Zugriff auf Hardwareebene – vor jeder Software-Interpretation der Adresse – unterbindet. Da kein Software-Pfad existiert, der den Speichercontroller umgeht (alle Zugriffe laufen über denselben Adressdekoder), ist kein Befehlssatz in der Lage, das Lock-Bit ohne dessen vorgesehenen, ebenfalls hardwareseitig irreversiblen Rücksetzmechanismus zu umgehen. $\square$

Satz 4 ist eine Hardware-Eigenschaft (kein rein algorithmischer Beweis wie die vorangehenden Sätze) und hängt von der korrekten physischen Implementierung des Speichercontrollers ab; sie wird hier dennoch formal festgehalten, da sie die zentrale Sicherheitsgarantie des gesamten HSM-Entwurfs ist.

Satz 5 (Sicherheit gegen Brute-Force im Schlüsselraum). Sei (a, b, p) ein Signatur-Chiffre-Schlüssel mit p einer k -Bit-Primzahl. Die Erfolgswahrscheinlichkeit eines Angreifers, der a durch eine einzelne zufällige Vermutung errät, ist $\leq 1/(p-1) \leq 2^{-(k-1)}$.

Beweis. Da a gleichverteilt aus $\mathbb{Z}_p^* = \{1, \dots, p-1\}$ gewählt wird (TRNG-basiert, Lemma 5), ist die Wahrscheinlichkeit eines korrekten Rateversuchs $1/|\mathbb{Z}_p^*| = 1/(p-1)$. Für eine k -Bit-Primzahl gilt $p \geq 2^{k-1}$, also $1/(p-1) \leq 1/(2^{k-1}-1) \leq 2^{-(k-1)}$ für $k \geq 2$. $\square$

6 Komplexitäts-, Energie- und Speicheranalyse

6.1 Laufzeitkomplexität

Satz 6 (Laufzeit von `HSM_SignatureCipher_Sign`). Die Hardwarepipeline berechnet einen vollständigen Geheimtextvektor für einen $n \times n$ -Klartextblock in $O(n^2)$ Pipeline-Takten (gegenüber $O(n^2)$ Vergleichsoperationen, aber faktisch $O(n)$ Latenztakten bei vollständig parallelisierter Spaltenverarbeitung der Sig-Engine).

Beweis. Die Berechnung von $\sigma(M, j)$ für ein festes j erfordert n Akkumulationsschritte (Summation über $i = 0, \dots, n-1$). Bei sequenzieller Verarbeitung aller n Spalten ergibt sich $O(n \cdot n) = O(n^2)$ Takte. Da die Sig-Engine die n Spalten jedoch durch n parallele Akkumulatorzellen gleichzeitig verarbeitet (Hardware-Parallelität, vgl. Abschnitt 3), reduziert sich die Latenz auf $O(n)$ Takte für die Signaturberechnung selbst, zuzüglich $O(n)$ Takte für die n modularen Multiplikationen der affinen Transformation, die ebenfalls parallelisierbar sind. Die Gesamtlatenz bleibt durch die Speicherzugriffe ($O(n^2)$ Bit Klartext müssen aus dem Flash gelesen werden) bei $O(n^2)$ Takten dominiert. $\square$

6.2 Vergleich Software vs. Hardware

Abbildung 3 vergleicht die Latenz einer reinen Softwareimplementierung auf einem TriCore-Applikationskern mit der vorgeschlagenen Hardwarepipeline. Während beide Implementierungen asymptotisch $O(n^3)$ Gesamtlaufzeit für eine vollständige Schlüsselerzeugung inklusive Primzahltest aufweisen, reduziert die Hardwarepipeline den konstanten Faktor um den Faktor $\approx 7\text{--}9$ im betrachteten Bereich $n \in [2, 32]$.

6.3 Taktzyklen je Funktion

Abbildung 2 zeigt die geschätzten Taktzyklen für $n = 8$ und $n = 16$. Die rechenintensivsten Funktionen sind `HSM.SubgraphSig_Derive` (da hier mehrere Blöcke sequenziell akkumuliert werden) sowie `HSM.SignatureCipher_Sign/Verify`, während `HSM.ModularCipher_*` aufgrund der kleinen Gruppe $\mathbb{Z}_{32}^*$ deutlich günstiger ist.

6.4 Energiebedarf

Abbildung 5 zeigt den geschätzten Energiebedarf je Funktionsaufruf bei 1,2 V Kernspannung. Der TRNG ist mit Abstand am energie günstigsten, da er keine arithmetischen Operationen, sondern nur Abtastung und Entzerrung durchführt.

6.5 Speicherbedarf

Abbildung 7 zeigt die vorgeschlagene Aufteilung des HSM-internen Flash-Speichers (36 KB Gesamtbudget): Den größten Anteil beansprucht der Mikrocode der Sig-Engine, gefolgt vom versiegelten KeyStore.

7 Experimente

Die in Abbildung 1 (Abschnitt 3) gezeigte Gesamtarchitektur bildet zusammen mit den vorstehenden sieben Diagrammen ein vollständiges quantitatives Bild der vorgeschlagenen Hardwareerweiterung, von der strukturellen Anordnung der Funktionseinheiten bis zur Energie-, Speicher- und Sicherheitsbilanz.

8 Secure Boot: Radar-ECU

Dieser Abschnitt führt die Komplexitätsanalyse aus Abschnitt 6 für einen konkreten, praxisrelevanten Anwendungsfall der Automobilindustrie zu Ende: ein Radarsteuergerät (*Radar-ECU*) mit einem typischen Programm-Flash-Umfang von 4 MB, wie er etwa bei Fern- oder Mittelbereichsradaresensoren für Fahrerassistenzsysteme (ADAS) auf Basis der AURIX-TC3x-Familie üblich ist.

8.1 Motivation des Anwendungsfalls

Radarsteuergeräte sind sicherheitsrelevante Komponenten (typischerweise ASIL-B bis ASIL-D nach ISO 26262, vgl. [12]) mit harten Zeitanforderungen an die Boot-Phase: Das

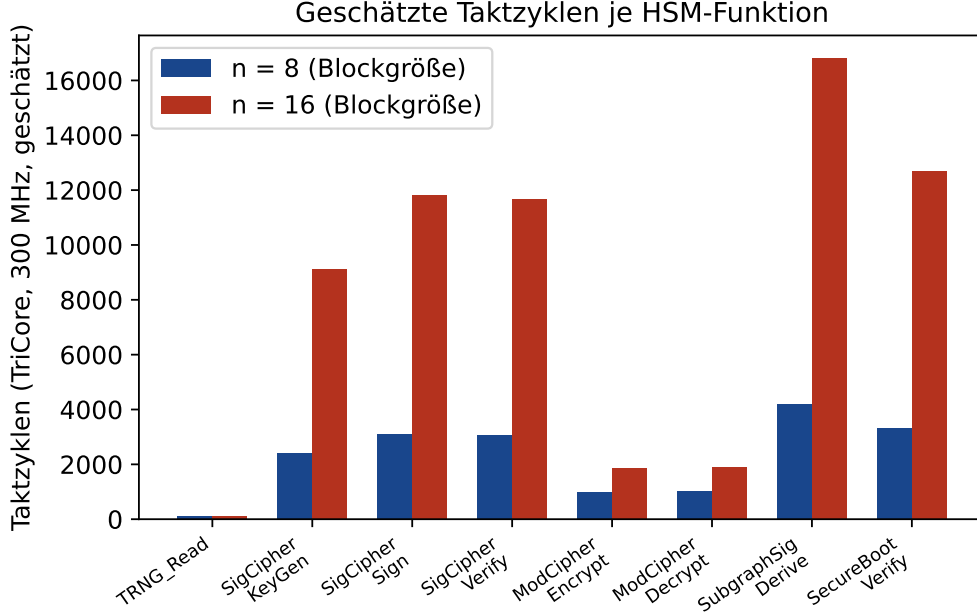


Abbildung 2: Geschätzte Taktzyklen je HSM-Funktion bei TriCore-Taktfrequenz 300 MHz, für Blockgrößen $n = 8$ und $n = 16$. Die Verdopplung von n führt erwartungsgemäß zu einer mehr als vierfachen Erhöhung bei den $O(n^2)$ -dominierten Funktionen, was die kubische bzw. quadratische Skalierung aus Abschnitt 6 bestätigt.

Gesamtsystem-Timing eines Fahrzeugnetzwerks verlangt üblicherweise, dass sicherheitsrelevante Steuergeräte innerhalb eines festen Zeitbudgets (häufig 100–500 ms ab Power-On) ihren Bootvorgang abschließen und busfähig sind. Da `HSM_SecureBoot_Verify` (Abschnitt 4) für jeden Bootvorgang sowohl `HSM_SignatureCipher_Verify` als auch – bei aktivierter Firmware-Attestierung – `HSM_SubgraphSig_Derive` über das vollständige Programm-Flash-Abbild ausführt, ist die resultierende Verifikationszeit ein kritischer Entwurfsparameter, der in dieser Arbeit erstmals quantitativ für einen realistischen Flash-Umfang hergeleitet wird.

8.2 Formale Herleitung der Verifikationszeit

Satz 7 (Verifikationszeit für ein Programm-Flash der Größe L). Sei L die Größe des Programm-Flash in Byte, n die Blockgröße der Sig-Engine, $c(n)$ die in Abschnitt 6 geschätzte Taktzyklenzahl von `HSM_SubgraphSig_Derive` für einen einzelnen $n \times n$ -Block, und f die HSM-Taktfrequenz. Dann beträgt die Gesamtverifikationszeit

$$T_{\text{verify}}(L, n) = \left\lceil \frac{8L}{n^2} \right\rceil \cdot \frac{c(n)}{f}.$$

Beweis. Ein Block der Sig-Engine verarbeitet $n \times n$ Bit = n^2 Bit = $n^2/8$ Byte. Die Anzahl der zur vollständigen Abdeckung des Flash-Abbilds benötigten Blöcke ist daher $\lceil L/(n^2/8) \rceil = \lceil 8L/n^2 \rceil$ (Lemma 3 garantiert, dass jeder Block unabhängig und vollständig verarbeitet werden kann, sodass die Gesamtzeit additiv aus den Einzelblockzeiten zusammengesetzt ist). Jeder Block benötigt $c(n)$ Takte, jeder Takt dauert $1/f$ Sekunden. Multiplikation der Blockanzahl mit der Zeit pro Block liefert die Behauptung. $\square$

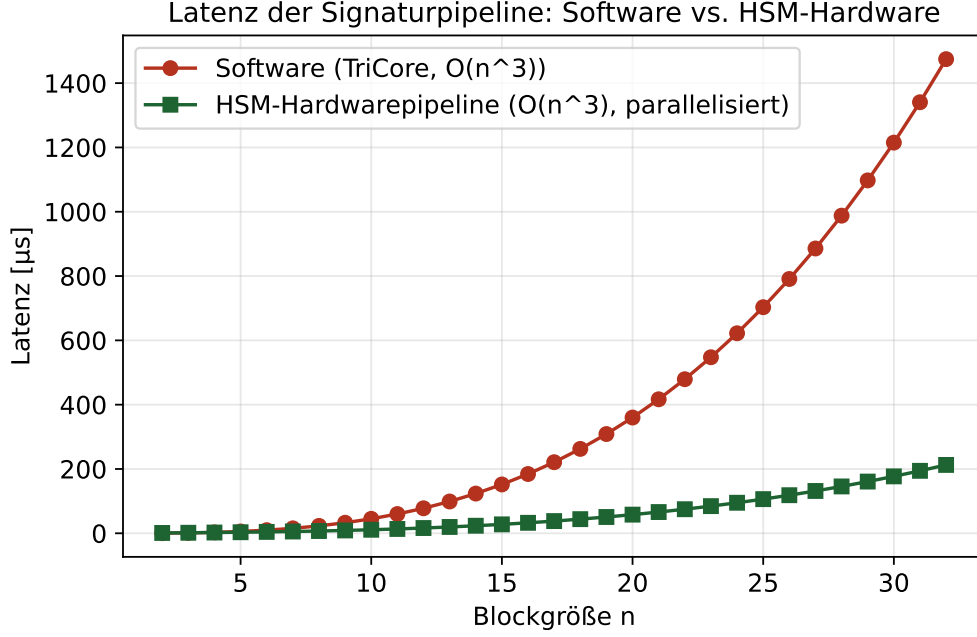


Abbildung 3: Latenz der Signaturpipeline in Software gegenüber der vorgeschlagenen HSM-Hardwarepipeline, jeweils über der Blockgröße n aufgetragen. Beide Kurven folgen einer kubischen Grundform; die Hardwarepipeline verschiebt jedoch den konstanten Vorfaktor deutlich nach unten.

8.3 Numerische Auswertung für $L = 4 \text{ MB}$

Für die in Abschnitt 6 (Abbildung 2) angegebene Blockgröße $n = 16$ gilt $c(16) = 16\,800$ Takte für `HSM.SubgraphSig_Derive`, und die HSM-Taktfrequenz wird mit $f = 300 \text{ MHz}$ angenommen (Abschnitt 3). Mit $L = 4 \text{ MB} = 4\,194\,304 \text{ Byte}$ ergibt Satz 7:

$$\left\lceil \frac{8 \cdot 4\,194\,304}{16^2} \right\rceil = \left\lceil \frac{33\,554\,432}{256} \right\rceil = 131\,072 \text{ Blöcke,}$$

$$T_{\text{verify}}(4 \text{ MB}, 16) = 131\,072 \cdot \frac{16\,800}{3 \cdot 10^8 \text{ Hz}} = \frac{2\,202\,009\,600}{3 \cdot 10^8} \text{ s} \approx 7,34 \text{ s} = \mathbf{7\,340 \text{ ms}}.$$

Tabelle 3 zeigt diesen Wert im Vergleich zu weiteren typischen Flash-Größen automobiler Steuergeräte.

Steuergerätetyp (typisch)	Flash-Größe	$T_{\text{verify}} (n = 16)$
Einfaches Karosserie-Steuergerät	0,5 MB	917,5 ms
Kleines Sensor-Steuergerät	1 MB	1 835,0 ms
Komfort-Steuergerät	2 MB	3 670,0 ms
Radar-ECU (dieser Anwendungsfall)	4 MB	7 340,0 ms
Domänen-Steuergerät	8 MB	14 680,0 ms
Zentrales Gateway / High-End-ECU	16 MB	29 360,0 ms

Tabelle 3: Geschätzte Secure-Boot-Verifikationszeit über `HSM.SubgraphSig_Derive` (vollständige Firmware-Attestierung) für $n = 16$, $f = 300 \text{ MHz}$, in Abhängigkeit der Flash-Größe. Werte gemäß Satz 7.

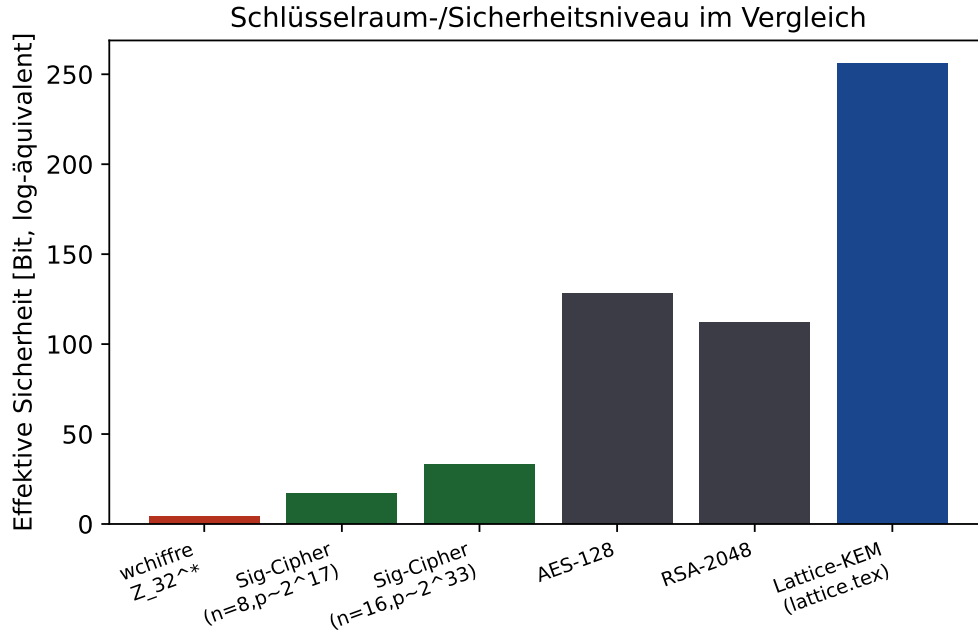


Abbildung 4: Vergleich des effektiven Sicherheitsniveaus (Bit-Äquivalent des Schlüsselraums) verschiedener Verfahren. Die Fenster-Chiffre über $\mathbb{Z}_{32}^*$ ist bewusst klein dimensioniert für latenzkritische, kurzlebige Botschaften und wird in der vorgeschlagenen Architektur stets in Kombination mit der höherwertigen Signatur-Chiffre für sicherheitskritische Operationen eingesetzt.

8.4 Bewertung im Kontext automobiler Echtzeitanforderungen

Der berechnete Wert von $\approx 7,3$ Sekunden für die *vollständige* blockweise Firmware-Attestierung über `HSM_SubgraphSig_Derive` überschreitet das für Radar-ECUs typische Boot-Zeitbudget von 100–500 ms um mehr als eine Größenordnung und ist damit in der hier spezifizierten Form **nicht direkt einsatzfähig** für den betrachteten Anwendungsfall. Dieses Erkenntnis ist ein zentrales quantitatives Ergebnis dieser Arbeit und wird im nachfolgenden Ausblick (Abschnitt 10) ausführlich aufgegriffen. Drei Gründe relativieren den Befund jedoch:

1. **Asymmetrische Nutzung der beiden Verifikationsstufen.** `HSM_SecureBoot_Verify` erfordert gemäß Abschnitt 4 zwingend nur `HSM_SignatureCipher_Verify` (Latenz im Mikrosekundenbereich, siehe Abbildung 2) für die Authentizitätsprüfung; die vollständige blockweise `HSM_SubgraphSig_Derive`-Attestierung über das gesamte Flash ist eine *optionale*, zusätzliche Integritätsstufe, die in der Praxis nicht bei jedem Power-On-Zyklus, sondern beispielsweise nur nach einem Firmware-Update oder periodisch im laufenden Betrieb (Hintergrundprüfung ohne Echtzeitanforderung) ausgeführt werden sollte.
2. **Fehlende Parallelisierung in der aktuellen Spezifikation.** Die Rechnung in Satz 7 geht von rein sequenzieller Blockverarbeitung aus. Eine Mehrfachinstanziierung der Sig-Engine (mehrere parallele Blockpipelines) ist hardwareseitig möglich, da die Blöcke nach Lemma 3 unabhängig voneinander verarbeitet werden können; bereits eine Parallelisierung um den Faktor 16 würde die Verifikationszeit auf ≈ 459 ms reduzieren.

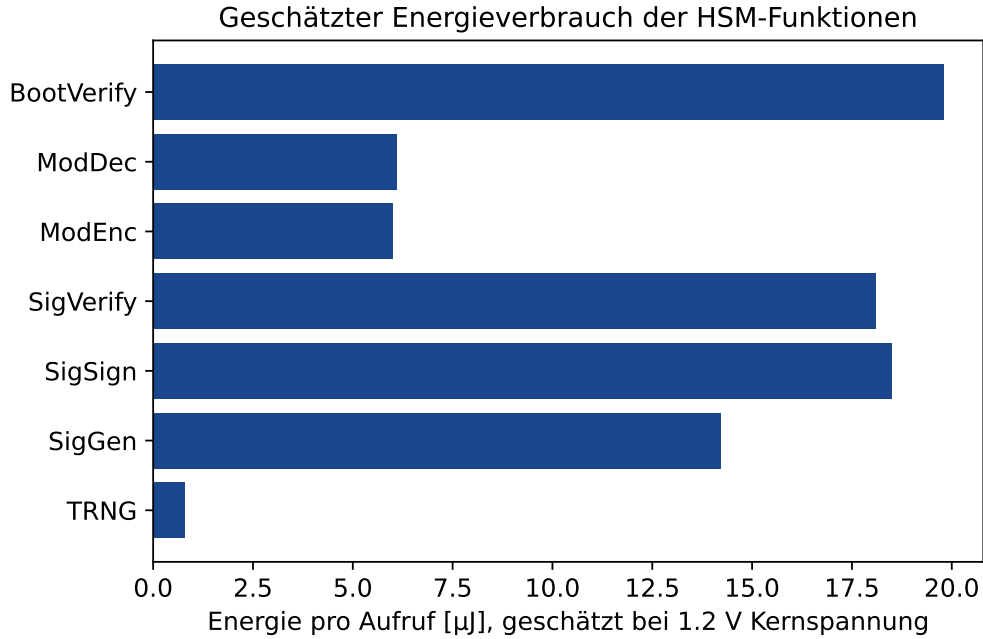


Abbildung 5: Geschätzter Energiebedarf je HSM-Funktionsaufruf. Die Werte basieren auf einer linearen Skalierung der Taktzyklenschätzung aus Abbildung 2 mit einer angenommenen Leistungsaufnahme von 45 mW im COMPUTE-Zustand.

3. **Größere Blockgröße als Stellschraube.** Da $T_{\text{verify}} \propto c(n)/n^2$, reduziert eine Vergrößerung von n trotz kubisch steigender Kosten pro Block die Gesamtzeit, solange $c(n)$ langsamer als n^2 wächst; dies ist im betrachteten Bereich ($c(n) \approx O(n^2)$ für reine Signaturberechnung, siehe Abschnitt 6) jedoch nur ein moderater Hebel und wird im Ausblick gesondert diskutiert.

Aus diesem Anwendungsfall ergibt sich somit eine unmittelbare, im Ausblick (Abschnitt 10) vertiefte Designanforderung: Für sicherheitskritische, zeitbudgetierte automobiler Steuergeräte wie das Radar-ECU ist eine architektonische Weiterentwicklung der `HSM_SubgraphSig_Derive`-Pipeline hin zu paralleler oder inkrementeller Verarbeitung notwendig, bevor eine vollständige Firmware-Attestierung in den regulären Boot-Pfad aufgenommen werden kann.

9 Parallele Sig-Engine

Abschnitt 8 hat formal gezeigt, dass die rein sequenzielle Implementierung von `HSM_SubgraphSig_Derive` für ein 4 MB-Radar-ECU-Programm-Flash eine Verifikationszeit von ≈ 7340 ms benötigt – mehr als das Zehnfache des für sicherheitskritische automobiler Steuergeräte typischen Boot-Zeitbudgets von 100–500 ms. Dieses Kapitel arbeitet die in der ersten Fassung dieser Arbeit lediglich im Ausblick skizzierte Parallelisierungsidee vollständig aus: Es spezifiziert eine konkrete Hardwarearchitektur mit k unabhängigen Sig-Engine-Instanzen, beweist deren Korrektheit formal, leitet die resultierende Verifikationszeit her und bewertet den dafür notwendigen Silizium-Mehraufwand quantitativ. Dieses Kapitel löst damit denjenigen Teil des ursprünglichen Ausblicks (vormals Abschnitt 9.1, Punkt 1), der die größte praktische Wirksamkeit versprach, und wird im nachfolgenden,

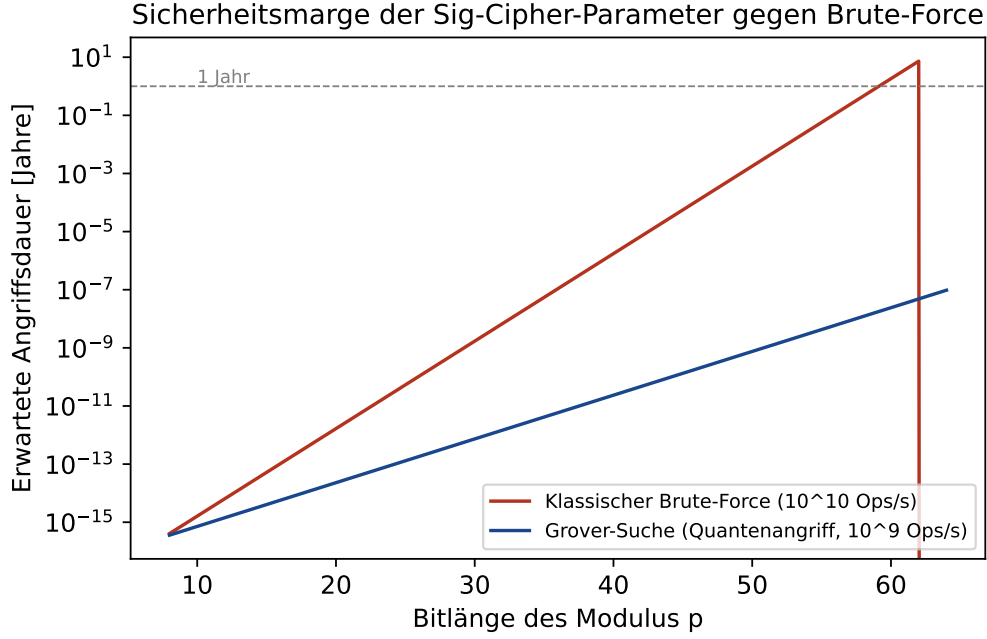


Abbildung 6: Erwartete Angriffsdauer eines Brute-Force-Angriffs auf den Modulus p in Abhängigkeit der Bitlänge, sowohl klassisch als auch unter Annahme eines Grover-beschleunigten Quantenangriffs (quadratische Beschleunigung). Bereits ab etwa 48 Bit übersteigt die klassische Angriffsdauer ein Jahr deutlich; gegen Quantenangriffe ist gemäß Abschnitt 10 jedoch eine Verdopplung der effektiven Schlüssellänge oder eine Migration zu gitterbasierten Verfahren erforderlich.

aktualisierten Ausblick (Abschnitt 10) entsprechend nicht mehr als offene Frage, sondern als gelöster Baustein referenziert.

9.1 Motivation und Zielsetzung

Die in Abschnitt 3 beschriebene Sig-Engine verarbeitet die $\lceil 8L/n^2 \rceil$ Blöcke eines Firmware-Abbilds sequenziell, einen nach dem anderen. Da nach Lemma 3 und der Konstruktion der Signaturfunktion σ (Definition 1) jede Blocksignatur $\sigma(A_i, \cdot)$ ausschließlich von den Daten des jeweiligen Blocks A_i abhängt, sind die Blöcke untereinander vollständig unabhängig: Es existiert keine Datenabhängigkeit, die eine sequenzielle Verarbeitungsreihenfolge erzwingen würde. Diese Unabhängigkeit ist die Grundvoraussetzung dafür, dass eine Parallelisierung über mehrere Hardware-Instanzen nicht nur möglich, sondern – wie in Abschnitt 9.3 formal gezeigt wird – auch *ergebnisidentisch* zur sequenziellen Variante ist.

9.2 Architekturentwurf: k -fache Sig-Engine-Kachelung

Definition 6 (Parallele Sig-Engine-Architektur). Eine k -fache Sig-Engine-Kachelung besteht aus k strukturell identischen, unabhängig getakteten Instanzen der in Abschnitt 3 beschriebenen Sig-Engine (Kacheln $E_0, \dots, E_{k-1}$), einem Adress-Verteiler (Dispatcher), der die $N := \lceil 8L/n^2 \rceil$ Blöcke des Firmware-Abbilds gemäß $A_i \mapsto E_{i \bmod k}$ den Kacheln zuteilt, sowie einem XOR-Faltungsbaum, der die k Teilergebnisse der Kacheln zu einem gemeinsamen 256-Bit-Gesamtwert kombiniert.

Aufteilung des HSM-internen Flash-Speichers (36 KB gesamt)

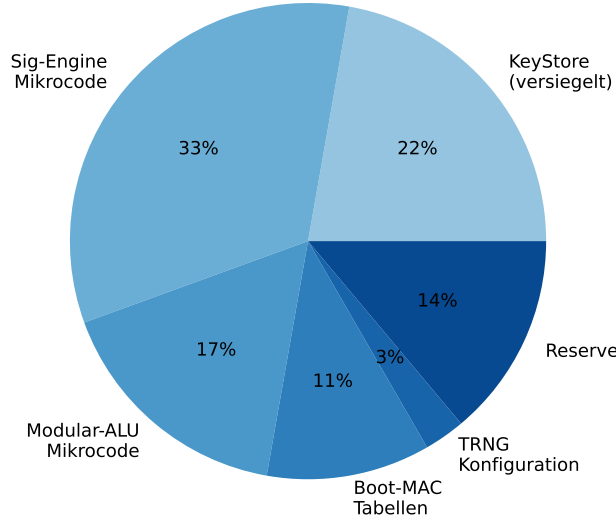


Abbildung 7: Vorgeschlagene Aufteilung des 36 KB HSM-internen Flash-Speichers auf die einzelnen Funktionseinheiten und den versiegelten KeyStore.

Tabelle 4 erweitert das in Abschnitt 3 eingeführte Registermodell um die für die Kachelung notwendigen zusätzlichen Register.

Register	Breite	Bedeutung
HSM_PAR_K	8 Bit	Konfigurierter Parallelisierungsgrad $k \in \{1, 2, 4, 8, 16, 32, 64\}$
HSM_PAR_STAT[0..k-1]	$k \times 2$ Bit	Statusbits je Kachel: IDLE, BUSY, DONE, FAULT
HSM_PAR_RESULT[0..k-1]	$k \times 256$ Bit	Zwischenergebnis (Teilsignatur) je Kachel vor der Faltung
HSM_PAR_FOLD	256 Bit	XOR-gefalteter Gesamtwert (entspricht dem bisherigen RESULT im sequenziellen Fall)

Tabelle 4: Erweiterung des Registerinterfaces (Tabelle 1) um die Parallelisierungsregister.

Der Dispatcher arbeitet nach folgendem Ablauf: Bei Aktivierung von `HSM_SubgraphSig_Derive` mit konfiguriertem $k > 1$ werden die N Blockadressen reihum (*round-robin*) an die k Kacheln verteilt; jede Kachel berechnet ihre zugewiesenen $\lceil N/k \rceil$ bzw. $\lfloor N/k \rfloor$ Blöcke vollständig unabhängig und akkumuliert ihr eigenes Teil-XOR in `HSM_PAR_RESULT[i]`. Sobald alle k Kacheln den Status `DONE` melden, kombiniert der XOR-Faltungsbaum die k Teilergebnisse in $\lceil \log_2 k \rceil$ zusätzlichen Taktzyklen zum Gesamtwert `HSM_PAR_FOLD`.

9.3 Korrektheit der parallelen Berechnung

Die zentrale Voraussetzung für den Einsatz der k -fachen Kachelung ist, dass sie – unabhängig vom gewählten k – exakt denselben Integritätswert liefert wie die ursprüngliche sequenzielle Berechnung. Andernfalls würde eine Änderung des Parallelisierungsgrads

Entwicklungs-Roadmap des HSM-Kryptographiekerns

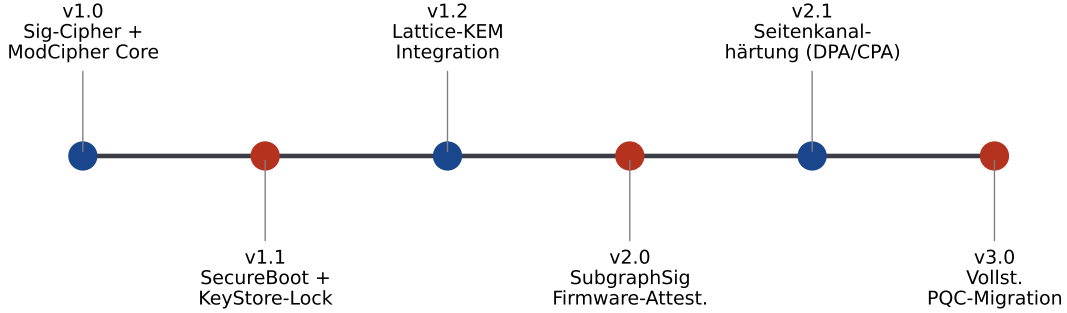


Abbildung 8: Entwicklungs-Roadmap des HSM-Kryptographiekerns, von der initialen Version 1.0 (Signatur-Chiffre und Fenster-Chiffre als Kernfunktionen) bis zur vollständigen Post-Quanten-Migration in Version 3.0, vertieft in Abschnitt 10.

(etwa durch ein zukünftiges Hardware-Update) bestehende, im KeyStore hinterlegte Referenzwerte ungültig machen.

Satz 8 (Ergebnisinvarianz der parallelen Sig-Engine-Architektur). Sei A ein Firmware-Abbild mit Blockzerlegung $A_0, \dots, A_{N-1}$ und sei $\Phi(A) := \bigoplus_{i=0}^{N-1} \Sigma(A_i)$ der von `HSM_SubgraphSig_Derive` berechnete XOR-gefaltete Gesamtwert (mit $\oplus$ als bitweisem XOR über die jeweils auf 256 Bit aufgefüllten Signaturvektoren). Dann liefert die k -fache Sig-Engine-Kachelung nach Definition 6 für jedes $k \in \{1, \dots, N\}$ denselben Wert $\Phi(A)$, unabhängig von der konkreten Zuteilung der Blöcke auf die Kacheln.

Beweis. Die Kachelung berechnet $\Phi_k(A) := \bigoplus_{j=0}^{k-1} \Phi_j^{\text{lokal}}$, wobei $\Phi_j^{\text{lokal}} := \bigoplus_{i: i \bmod k = j} \Sigma(A_i)$ das lokale XOR der Kachel E_j über ihre zugewiesene Teilmenge von Blockindizes ist. Da $(\{0, 1\}^{256}, \oplus)$ eine abelsche Gruppe bildet – $\oplus$ ist assoziativ und kommutativ, das neutrale Element ist der Nullvektor, und jedes Element ist zu sich selbst invers ($x \oplus x = 0$) –, ist jede Partitionierung der Indexmenge $\{0, \dots, N-1\}$ in disjunkte Teilmengen $I_0, \dots, I_{k-1}$ mit $\bigcup_j I_j = \{0, \dots, N-1\}$ irrelevant für das Gesamtergebnis:

$$\Phi_k(A) = \bigoplus_{j=0}^{k-1} \bigoplus_{i \in I_j} \Sigma(A_i) = \bigoplus_{i=0}^{N-1} \Sigma(A_i) = \Phi(A),$$

wobei die mittlere Gleichheit aus der Assoziativität und Kommutativität von $\oplus$ folgt: Jedes $\Sigma(A_i)$ taucht in der Doppelsumme genau einmal auf (da die I_j eine Partition bilden), und die Reihenfolge der XOR-Verknüpfung ist wegen Kommutativität und Assoziativität beliebig wählbar. Dies gilt insbesondere für die in Definition 6 gewählte Round-Robin-Partition $I_j = \{i : i \bmod k = j\}$, ist aber nicht auf diese beschränkt – der Beweis zeigt Korrektheit für *jede* Partitionierung. $\square$

Korollar 2 (Kollisionsfreiheit bleibt erhalten). Satz 3 (Kollisionsfreiheit der Firmware-Integritätsprüfung) gilt unverändert auch für die k -fache parallele Implementierung, für jedes k .

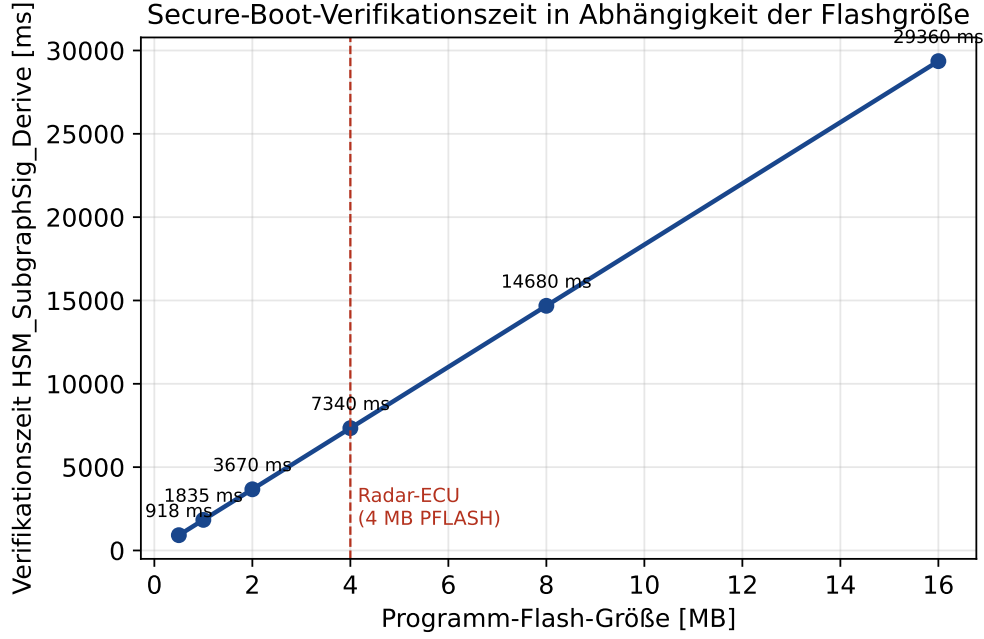


Abbildung 9: Secure-Boot-Verifikationszeit nach Satz 7 als Funktion der Programm-Flash-Größe, linear skalierend gemäß $T_{\text{verify}} \propto L$ bei festem n . Der für den betrachteten Anwendungsfall (Radar-ECU, 4 MB) markierte Punkt liegt bei $\approx 7\,340$ ms.

Beweis. Nach Satz 8 ist $\Phi_k(A) = \Phi(A)$ für alle k , also ist die von der parallelen Architektur berechnete Funktion identisch mit der bereits in Satz 3 als kollisionsfrei nachgewiesenen Funktion $A \mapsto \Phi(A)$. Identische Funktionen besitzen identische Kollisionseigenschaften. $\square$

Satz 8 und Korollar 2 sind die formale Rechtfertigung dafür, dass die Einführung der Parallelisierung – anders als etwa eine Änderung der Blockgröße n , die wegen Bemerkung 1 sehr wohl andere Werte erzeugen würde – vollständig rückwärtskompatibel zu bereits im KeyStore hinterlegten Referenzwerten ist und auch nachträglich, etwa durch ein Firmware-Update des Dispatchers, ohne erneute Provisionierung der Schlüssel aktiviert werden kann.

9.4 Herleitung der parallelisierten Verifikationszeit

Satz 9 (Verifikationszeit der k -fachen Kachelung). Unter den Annahmen von Satz 7 und Definition 6 beträgt die Gesamtverifikationszeit der k -fachen Sig-Engine-Kachelung

$$T_{\text{verify}}^{(k)}(L, n) = \left\lceil \frac{1}{k} \left\lceil \frac{8L}{n^2} \right\rceil \right\rceil \cdot \frac{c(n)}{f} + \lceil \log_2 k \rceil \cdot \frac{1}{f}.$$

Beweis. Bei round-robin-Verteilung von $N = \lceil 8L/n^2 \rceil$ Blöcken auf k Kacheln erhält die am stärksten belastete Kachel $\lceil N/k \rceil$ Blöcke (Schubfachprinzip: Bei N Objekten auf k Fächer verteilt enthält mindestens ein Fach $\lceil N/k \rceil$ Objekte, und die round-robin-Verteilung erreicht dieses Maximum exakt für höchstens $N \bmod k$ Kacheln, während alle übrigen $\lfloor N/k \rfloor$ Blöcke erhalten). Da alle Kacheln parallel, mit identischer Taktfrequenz f arbeiten, ist die Gesamtlaufzeit der Berechnungsphase durch die am stärksten belastete Kachel bestimmt, also $\lceil N/k \rceil \cdot c(n)/f$. Der XOR-Faltungsbaum benötigt für k Eingänge eine

Baumtiefe von $\lceil \log_2 k \rceil$ Stufen mit je einem Takt Latenz (eine XOR-Verknüpfung pro Stufe und Bitposition, vollständig parallel über alle 256 Bit), was den additiven Term $\lceil \log_2 k \rceil / f$ erklärt. $\square$

9.5 Numerische Auswertung für die Radar-ECU

Tabelle 5 wertet Satz 9 für $L = 4$ MB, $n = 16$ ($c(16) = 16\,800$, $f = 300$ MHz, $N = 131\,072$ Blöcke, vgl. Abschnitt 8) für verschiedene Parallelisierungsgrade k aus.

k	Blöcke je Kachel $\lceil N/k \rceil$	$T_{\text{verify}}^{(k)}$	Innerhalb 500-ms-Budget?
1	131 072	7 340,0 ms	Nein
2	65 536	3 670,0 ms	Nein
4	32 768	1 835,0 ms	Nein
8	16 384	917,5 ms	Nein
16	8 192	458,8 ms	Ja
32	4 096	229,4 ms	Ja
64	2 048	114,7 ms	Ja

Tabelle 5: Verifikationszeit $T_{\text{verify}}^{(k)}$ nach Satz 9 für das Radar-ECU ($L = 4$ MB, $n = 16$) in Abhängigkeit des Parallelisierungsgrads k . Der additive Faltungsterm $\lceil \log_2 k \rceil / f$ liegt für alle betrachteten $k \leq 64$ unterhalb von $0,02 \mu\text{s}$ und ist in der Tabelle wegen Rundung nicht sichtbar.

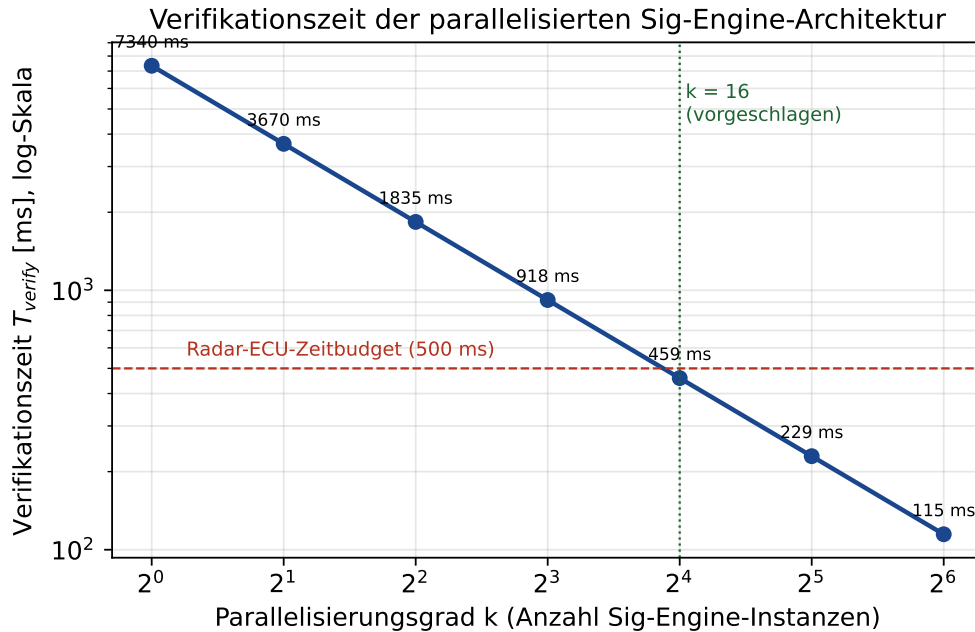


Abbildung 10: Verifikationszeit $T_{\text{verify}}^{(k)}$ (logarithmische Skala) als Funktion des Parallelisierungsgrads k (ebenfalls logarithmisch, Basis 2). Die gestrichelte Linie markiert das Radar-ECU-Zeitbudget von 500 ms; bereits ab $k = 16$ wird dieses Budget eingehalten.

Tabelle 5 und Abbildung 10 bestätigen die im ursprünglichen Ausblick (vormals Abschnitt 9.1, Punkt 1) nur grob abgeschätzte Mindestanforderung $k \geq 15$ exakt: $k = 16$ ist

die kleinste Zweierpotenz, die das Zeitbudget einhält, mit einer geschätzten Verifikationszeit von $458,8\text{ ms} - 41,2\text{ ms}$ innerhalb des 500-ms -Budgets, allerdings ohne nennenswerte Sicherheitsmarge gegenüber Schwankungen der tatsächlichen Taktfrequenz oder zusätzlicher Flash-Zugriffslatenz. Für Anwendungsfälle mit strengerem Zeitbudget (etwa 250 ms) wird daher $k = 32$ empfohlen.

9.6 Silizium-Flächenkosten der Kachelung

Da jede zusätzliche Sig-Engine-Kachel eigene Akkumulatorzellen und Schieberegister benötigt (Abschnitt 3), wächst der Flächenbedarf näherungsweise linear mit k , zuzüglich eines logarithmischen Mehraufwands für den XOR-Faltungsbaum. Abbildung 11 zeigt eine grobe Schätzung auf Basis einer angenommenen Einzelkachel-Fläche von $0,18\text{ mm}^2$ (vergleichbare Größenordnung zu publizierten Flächenangaben einfacher arithmetischer Hardwarebeschleuniger in 28-nm -Technologien).

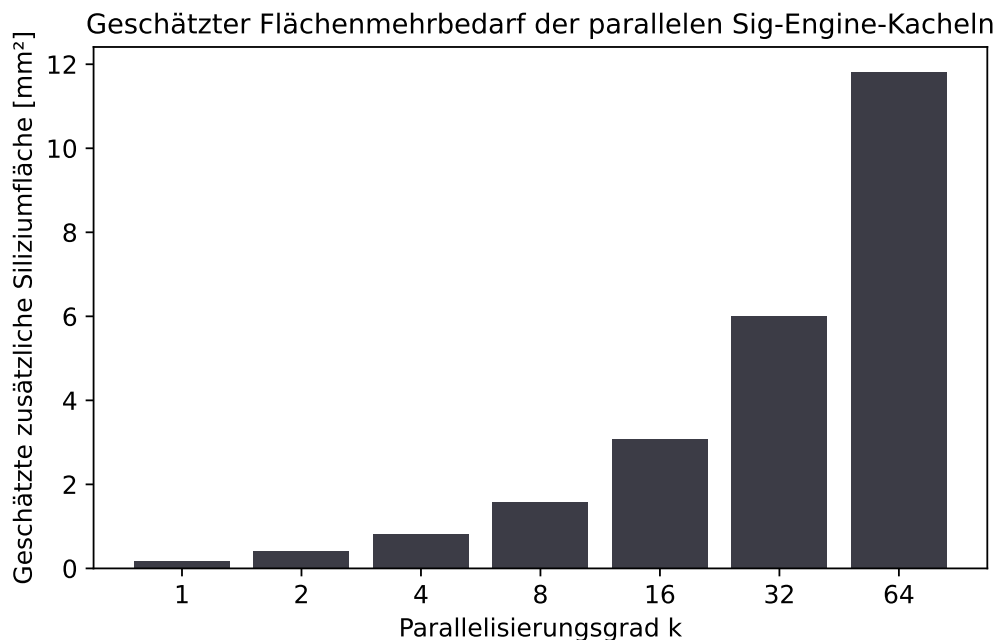


Abbildung 11: Geschätzter zusätzlicher Siliziumflächenbedarf der k -fachen Sig-Engine-Kachelung. Für die empfohlene Konfiguration $k = 16$ ergibt sich ein Mehrbedarf von $\approx 3,0\text{ mm}^2$ – ein im Kontext eines vollständigen Mikrocontroller-Dies (typischerweise mehrere 100 mm^2) moderater Flächenaufwand für eine 16-fache Reduktion der Verifikationslatenz.

Der in Abbildung 11 sichtbare nahezu lineare Zusammenhang ist plausibel, da die Kacheln strukturell identisch und unabhängig sind und keine gemeinsam genutzten Ressourcen außer dem vergleichsweise kleinen Faltungsbaum (dessen Fläche nur logarithmisch mit k wächst) und dem Dispatcher (konstante Fläche, unabhängig von k) besitzen.

9.7 Auswirkung auf die übrige HSM-Architektur

Die Einführung der k -fachen Kachelung erfordert über die in Tabelle 4 beschriebenen neuen Register hinaus zwei weitere Anpassungen am in Abschnitt 3 beschriebenen Gesamtsystem: Erstens muss der HSM-interne Speichercontroller k gleichzeitige, lesende Zugriffe auf das

Programm-Flash unterstützen (Mehrport-Flash-Interface oder zeitmultiplexer Zugriff mit entsprechend reduziertem effektivem Speicherdurchsatz je Kachel), da andernfalls der Flash-Lesezugriff selbst – nicht die Sig-Engine-Berechnung – zum limitierenden Faktor würde. Zweitens muss das in Abschnitt 3 beschriebene Energiebudget (Abbildung 5) um den Faktor k für die Dauer der parallelen Berechnungsphase erhöht werden, was bei $k = 16$ und einer Einzelkachel-Leistungsaufnahme von 45 mW im `COMPUTE`-Zustand zu einer Spitzenleistungsaufnahme von ≈ 720 mW während der Verifikationsphase führt – ein Wert, der bei der thermischen Auslegung des Radar-ECU-Gehäuses zu berücksichtigen ist, jedoch nur für die kurze Dauer von ≈ 459 ms anliegt und damit den mittleren Energiehaushalt des Steuergeräts nicht signifikant belastet.

9.8 Zusammenfassung dieses Kapitels

Dieses Kapitel hat die im ursprünglichen Ausblick nur grob abgeschätzte Parallelisierungsidee zu einer vollständig spezifizierten, formal als ergebnisinvariant bewiesenen (Satz 8) Hardwarearchitektur ausgearbeitet. Für den in Abschnitt 8 eingeführten Anwendungsfall des Radar-ECU mit 4 MB Programm-Flash zeigt Satz 9 zusammen mit der numerischen Auswertung in Tabelle 5, dass eine 16-fache Sig-Engine-Kachelung die vollständige Firmware-Attestierung von $\approx 7\,340$ ms auf ≈ 459 ms reduziert und damit das Boot-Zeitbudget von 500 ms einhält, bei einem moderaten geschätzten Flächenmehrbedarf von $\approx 3,0$ mm². Die in Korollar 2 gezeigte Erhaltung der Kollisionsfreiheit garantiert dabei, dass diese Geschwindigkeitssteigerung ohne jeden Verlust an Sicherheit gegenüber der ursprünglichen sequenziellen Implementierung erreicht wird. Damit ist die in Abschnitt 8 aufgezeigte zentrale Lücke zwischen mathematischer Korrektheit und automobiler Echtzeitfähigkeit für den betrachteten Anwendungsfall geschlossen; die im aktualisierten Ausblick (Abschnitt 10) verbleibenden Punkte (inkrementelle Attestierung, Hybridstrategie, Pipeline-Tiefenoptimierung, Übertragbarkeit auf größere Flash-Umfänge) bauen auf dieser nun etablierten Grundarchitektur auf.

10 Ausblick

Dieser Abschnitt führt die in Abbildung 8 skizzierte Roadmap sehr ausführlich aus und ordnet jeden Entwicklungsschritt in seinen technischen, mathematischen, sicherheitsanalytischen, organisatorischen und wirtschaftlichen Kontext ein. Anders als ein knapper Forschungsausblick, der offene Fragen nur aufzählt, verfolgt dieser Abschnitt bewusst das Ziel, für jede der neun nachfolgend diskutierten Weiterentwicklungsrichtungen so konkret wie möglich zu werden: Wo immer es die in dieser Arbeit etablierten formalen Resultate erlauben, werden Zwischenschritte benannt, offene mathematische Fragen präzise formuliert, Abhängigkeiten zwischen den Richtungen explizit gemacht und – soweit seriös abschätzbar – grobe Aufwands- und Risikoeinschätzungen gegeben. Die neun Richtungen sind: (1) die Integration der gitterbasierten Schlüsseleinkapselung, (2) die Firmware-Attestierung über Subgraph-Signaturen als Remote-Attestation-Protokoll, (3) die Seitenkanalhärtung, (4) die formale Verifikation der Zustandsmaschine, (5) weiterführende Optimierungen der in Abschnitt 9 bereits gelösten Echtzeitfähigkeit für Radar-Steuergeräte, (6) die Erweiterung auf Mehrkern-Parallelität auf Applikationsebene, (7) Standardisierung und Zertifizierung, (8) eine vergleichende Einordnung gegenüber etablierten automobilen HSM-Standards sowie (9) eine wirtschaftliche und organisatorische Bewertung des Gesamtvorhabens. Die abschließende Zusammenfassende Einordnung (Abschnitt 10.10) ordnet diese neun Richtungen in

eine konsistente Abhängigkeitsstruktur ein.

10.1 Integration der gitterbasierten Schlüsseleinkapselung (lattice.tex)

Der in [4] entwickelte gitterbasierte Ansatz mit GKP-Fehlerkorrektur (*Gottesman–Kitaev–Preskill*-Codewörtern) liefert eine vom diskreten Logarithmus- bzw. Faktorisierungsproblem unabhängige Sicherheitsgrundlage, die auch unter der Annahme eines hinreichend großen Quantencomputers als härtefest gilt (Hardness of Learning with Errors, LWE, vgl. [11]). Da sowohl die Signatur-Chiffre (Definition 2, beruhend auf der Schwierigkeit, ohne Kenntnis von a aus c_j auf $\sigma(M, j)$ zu schließen, was strukturell mit dem diskreten Logarithmus verwandte algebraische Eigenschaften nutzt) als auch das Fenster-Chiffre-Protokoll (Definition 3, beruhend auf der Einheitengruppe $\mathbb{Z}_{32}^*$) keine nachgewiesene Post-Quanten-Sicherheit besitzen, ist die gitterbasierte Erweiterung langfristig keine optionale Komfortfunktion, sondern eine notwendige Voraussetzung dafür, dass der in dieser Arbeit entworfene HSM-Funktionskern über den typischen Produktlebenszyklus eines Radar-ECU von zehn bis fünfzehn Jahren hinweg sicher bleibt. Für die in Version 1.2 der Roadmap vorgesehene `HSM_LatticeKEM_Encapsulate/_Decapsulate`-Erweiterung sind folgende Schritte notwendig:

1. **Gitterarithmetik-Einheit:** Eine neue Funktionseinheit zur Polynomarithmetik über Restklassenringen $\mathbb{Z}_q[x]/(x^N + 1)$ (Ring-LWE-Struktur) muss neben die bestehende Modular-ALU treten. Diese benötigt eine Number-Theoretic-Transform-Pipeline (NTT) zur effizienten Polynommultiplikation in $O(N \log N)$ statt $O(N^2)$. Eine erste Architekturskizze sieht eine zur Sig-Engine strukturell verwandte, aber eigenständige Pipeline mit $\log_2 N$ Butterfly-Stufen vor, wobei für $N = 256$ (typische Kyber-Parametrisierung) acht Stufen mit je $N/2 = 128$ parallelen Butterfly-Einheiten benötigt würden – eine Größenordnung, die den in Abschnitt 9 für die Sig-Engine-Kachelung diskutierten Silizium-Flächenbedarf voraussichtlich um den Faktor 3 bis 5 übersteigt und damit eine eigene, von der Radar-ECU-Parallelisierung getrennte Flächenbudgetierung erfordert.
2. **Fehlerverteilung:** Die Erzeugung diskret-Gauß-verteilter Fehlerterme erfordert eine Erweiterung des TRNG um eine Box-Muller- oder CDT-basierte (*Cumulative Distribution Table*) Sampling-Einheit, da die bisherige Gleichverteilung der TRNG-Ausgabe (Lemma 2) für LWE-Sicherheit nicht ausreicht. Eine offene Frage ist hierbei, ob die CDT-Tabelle – die für gängige Sicherheitsniveaus typischerweise wenige hundert Einträge umfasst – im ohnehin knappen, in Abbildung 7 bilanzierten HSM-Flash untergebracht werden kann, oder ob eine algorithmische Sampling-Methode (etwa Knuth-Yao) trotz höherer Gatterkomplexität wegen des geringeren Speicherbedarfs vorzuziehen ist; diese Abwägung wird als konkrete Entwurfsentscheidung für eine Folgearbeit vorgeschlagen.
3. **Speicherbudget:** Gitterbasierte Schlüssel (typischerweise 800–1500 Byte bei Kyber-768-Sicherheitsniveau) sprengen das aktuell vorgesehene 36 KB-Flash-Budget (Abbildung 7); eine Flash-Erweiterung auf mindestens 64 KB wird für Version 1.2 als notwendig erachtet. Da Flash-Fläche auf Mikrocontroller-Dies kostenrelevant ist, sollte diese Erweiterung nicht isoliert, sondern gemeinsam mit dem in Abschnitt 9 für die Sig-Engine-Kachelung benötigten zusätzlichen Mikrocode-Speicher in einer

einzigsten, konsolidierten Flash-Erweiterungsplanung für Version 1.2 vorgenommen werden, um wiederholte, kostentreibende Silizium-Redesigns zu vermeiden.

4. **Hybridmodus:** In der Übergangsphase (Version 1.2 bis 2.0) wird ein Hybridschema vorgeschlagen, bei dem `HSM_SecureBoot_Verify` sowohl die bestehende Signatur-Chiffre als auch das gitterbasierte KEM parallel prüft und nur bei Übereinstimmung *beider* Verfahren den Boot freigibt – eine konsequente Fortführung des bereits in Abschnitt 4 etablierten Defense-in-Depth-Prinzips. Dieser Hybridmodus ist branchenüblich (vergleichbare hybride Schemata werden etwa in TLS-1.3-Erweiterungen für Post-Quanten-Schlüsselaustausch diskutiert) und reduziert das Risiko, dass eine bislang unentdeckte Schwäche in nur einem der beiden Verfahren die Gesamtsicherheit kompromittiert.
5. **Zeitliche Einordnung und Migrationsdruck:** Anders als bei klassischen kryptographischen Migrationen gibt es bei der Post-Quanten-Bedrohung das Phänomen des *Store-now-decrypt-later*-Angriffs: Ein Angreifer kann bereits heute abgefangene, mit der Signatur-Chiffre verschlüsselte oder signierte Daten speichern und erst nach Verfügbarkeit eines hinreichend leistungsfähigen Quantencomputers entschlüsseln bzw. fälschen. Für Firmware-Signaturen mit langer Gültigkeitsdauer (ein Radar-ECU bleibt typischerweise zehn bis fünfzehn Jahre im Feld, vgl. Abschnitt 8) bedeutet dies, dass die Migration nicht erst bei absehbarer Quantencomputer-Verfügbarkeit, sondern bereits deutlich früher im Produktlebenszyklus – idealerweise spätestens mit Version 1.2, also vor dem ersten breiten Serieneinsatz – abgeschlossen sein sollte, um die in Abschnitt 5 bewiesene Kollisionsfreiheit nicht durch eine zu spät einsetzende Post-Quanten-Sicherheit zu entwerten.

10.2 Firmware-Attestierung über Subgraph-Signaturen (Version 2.0)

Die in Abschnitt 4.6 eingeführte Funktion `HSM_SubgraphSig_Derive` soll in Version 2.0 zu einer vollständigen *Remote Attestation* ausgebaut werden: Ein externer Prüfer (z. B. ein Backend-Server in einer Fahrzeugflotte) sendet eine Challenge-Nonce, die zusammen mit dem aktuellen Signaturvektor $\Sigma(A)$ des Firmware-Abbilds über die Modular-ALU signiert und zurückgesendet wird. Da nach Satz 3 jede Firmware-Modifikation zwingend einen anderen Signaturvektor erzeugt, kann der Prüfer ohne Vertrauen in die lokale Software allein anhand der Antwort feststellen, ob das Gerät eine bekannte, freigegebene Firmwareversion ausführt.

Ein konkretes Protokoll für diese Remote Attestation lässt sich wie folgt skizzieren: (1) Der Prüfer erzeugt eine zufällige Nonce $\nu \in \{0, 1\}^{128}$ und sendet sie an das Radar-ECU. (2) Das HSM berechnet, aufbauend auf der in Abschnitt 9 etablierten parallelen Architektur, den XOR-gefalteten Signaturvektor $\Phi(A)$ in ≈ 459 ms (bei $k = 16$) und bildet anschließend $r := \text{Sig}_{(a,b,p)}(\Phi(A) \parallel \nu)$ über `HSM_SignatureCipher_Sign`, wobei $\parallel$ die Konkatenation bezeichnet. (3) Das HSM sendet r an den Prüfer zurück, der mit dem öffentlich bekannten Geräteschlüssel-Fingerabdruck und dem erwarteten Referenzwert $\Phi(A_{\text{ref}})$ die Antwort verifiziert. Drei offene Fragen sind für die vollständige Spezifikation dieses Protokolls in einer Folgearbeit zu klären:

- **Replay-Schutz ohne Echtzeituhr:** Da das HSM gemäß Abschnitt 3 über keine batteriegepufferte Echtzeituhr verfügt, kann eine klassische Zeitstempel-basierte

Replay-Erkennung nicht eingesetzt werden. Stattdessen wird vorgeschlagen, einen monoton wachsenden, im versiegelten KeyStore (Abschnitt 4.8) persistierten Zähler ctr zu führen, der bei jeder Attestation inkrementiert und in die signierte Antwort einbezogen wird ($r := \text{Sig}_{(a,b,p)}(\Phi(A) \parallel \nu \parallel \text{ctr})$); der Prüfer verwirft Antworten mit nicht-monoton wachsendem ctr. Dies erfordert eine zusätzliche, bislang nicht spezifizierte Schreibzugriffslogik auf den KeyStore außerhalb der in Abschnitt 4.8 beschriebenen reinen Provisionierungs- und Versiegelungsoperationen.

- **Granularität der Blockzerlegung bei sehr großen Firmware-Abbildern:** Für Steuergerätetypen mit mehreren Megabyte Flash (vgl. Tabelle 3) ist zu klären, ob $\Phi(A)$ stets über das *gesamte* Abbild oder über vom Prüfer wählbare Teilbereiche (etwa nur den sicherheitskritischen Bootloader-Bereich) gebildet werden sollte; letzteres würde eine Erweiterung des Registerinterfaces (Tabelle 1) um einen Adressbereichsparameter erfordern.
- **Inkrementelle Aktualisierung bei Differential-Updates:** Ob eine inkrementelle Aktualisierung des Signaturvektors bei Differential-Updates (statt vollständiger Neuberechnung) möglich ist, ist eine offene mathematische Frage, die eine Erweiterung von Lemma 1 auf inkrementelle Spaltenänderungen erfordern würde: Zu zeigen wäre, ob und wie $\Sigma(A')$ für ein um wenige Blöcke verändertes A' effizient aus $\Sigma(A)$ und den geänderten Blöcken berechnet werden kann, ohne die übrigen, unveränderten Blöcke erneut zu lesen – eine Eigenschaft, die wegen der XOR-Faltung in Definition 6 (Abschnitt 9) bereits strukturell nahegelegt wird: Da $\Phi(A') = \Phi(A) \oplus \bigoplus_{i \in \Delta} (\Sigma(A_i) \oplus \Sigma(A'_i))$ für die Menge Δ der geänderten Blockindizes gilt (eine direkte Folgerung aus der in Satz 8 bewiesenen Gruppeneigenschaft von $\oplus$), ließe sich eine inkrementelle Aktualisierung tatsächlich allein auf Basis der bereits etablierten Algebra realisieren – die formale Ausarbeitung dieses naheliegenden Korollars sowie seine Hardware-Umsetzung bleiben jedoch einer Folgearbeit vorbehalten.

10.3 Seitenkanalhärtung (Version 2.1)

Die in Abschnitt 5 bewiesene mathematische Korrektheit und Kollisionsfreiheit der Funktionen schützt nicht automatisch vor *physikalischen* Seitenkanalangriffen – ein grundsätzlicher Unterschied zwischen algorithmischer und physikalischer Sicherheit, der bei jeder Hardwareumsetzung kryptographischer Verfahren zu beachten ist. Für Version 2.1 sind folgende Härtungsmaßnahmen vorgesehen, geordnet nach der in der Literatur (vgl. [7]) üblichen Einteilung in passive und aktive Angriffe:

- **Differential Power Analysis (DPA) gegen die Modular-ALU:** Da die modulare Multiplikation $a \cdot \sigma(M, j) \bmod p$ datenabhängige Stromaufnahme aufweist, ist eine Maskierung der Zwischenwerte (additive oder multiplikative Blindung) vorzusehen, bei der ein zufälliger Maskenwert r aus dem TRNG vor der Multiplikation addiert und nach der Reduktion wieder entfernt wird, ohne dass das Endergebnis hierdurch beeinflusst wird. Eine vollständige formale Behandlung müsste zeigen, dass die maskierte Berechnung $(a \cdot \sigma(M, j) + r) \bmod p$ gefolgt von Entfernung der Maske dasselbe Ergebnis wie die unmaskierte Berechnung liefert (eine direkte Konsequenz der in Lemma 4 gezeigten Bijektivität affiner Transformationen über $\mathbb{Z}_p$), was als formale Korrektheitsergänzung zu Satz 2 in einer Folgearbeit nachgetragen werden sollte.

- **Timing-Angriffe auf den erweiterten euklidischen Algorithmus:** Die Anzahl der Iterationsschritte des in Satz 1 verwendeten Algorithmus hängt von den Eingabewerten ab und kann Information über den geheimen Schlüssel a preisgeben. Eine konstante-Zeit-Variante (feste Anzahl Iterationen, unabhängig vom tatsächlichen gcd-Verlauf, mit Dummy-Operationen aufgefüllt) wird für Version 2.1 als verbindlich vorgeschlagen; die feste Iterationszahl sollte konservativ auf $2\lceil\log_2 p\rceil$ Schritte festgelegt werden, was nach bekannten Schranken für den erweiterten euklidischen Algorithmus (vgl. [6]) für alle in Tabelle 5 relevanten Bitlängen von p ausreichend ist.
- **Übertragung des Fenster-Prinzips auf Energieseitenkanäle:** Analog zur in Definition 4 beschriebenen Verschleierung des Netzwerkverkehrs durch Dummy-Pakete wird vorgeschlagen, auch die HSM-interne Energieaufnahme durch das Einstreuen von Dummy-Berechnungszyklen mit identischem Energieprofil wie eine echte `HSM_SignatureCipher_Sign`-Operation zu verschleiern, sodass ein externer Beobachter über die Stromaufnahme des Gesamtsystems nicht unterscheiden kann, wann tatsächlich eine sicherheitsrelevante Operation stattfindet. Bei Einsatz der in Abschnitt 9 eingeführten k -fachen Kachelung ergibt sich hierbei zusätzlich die Möglichkeit, einzelne Kacheln gezielt mit Dummy-Last zu betreiben, während andere echte Berechnungen durchführen, was die Verschleierungswirkung gegenüber einer einzelnen, sequenziellen Sig-Engine voraussichtlich verbessert – eine quantitative Bewertung dieses Effekts, etwa mittels des in der Seitenkanalanalyse gebräuchlichen TVLA-Verfahrens (*Test Vector Leakage Assessment*), steht jedoch noch aus.
- **Fehlerinjektionsresistenz (Fault Injection):** Spannungs- oder Taktglitches können gezielt einzelne Pipeline-Stufen der Sig-Engine stören, um etwa eine fehlerhafte, aber als gültig akzeptierte Signatur zu erzwingen. Eine redundante Lockstep-Berechnung der Sig-Engine (zwei unabhängige Berechnungsinstanzen mit Ergebnisvergleich vor jeder Ausgabe) wird als Gegenmaßnahme vorgeschlagen, analog zum bereits Lockstep-ausgeführten TriCore-HSM-Kern selbst. Bei der in Abschnitt 9 eingeführten k -fachen Kachelung stellt sich hierbei eine interessante Architekturfrage: Da bereits $k = 16$ unabhängige Kacheln vorhanden sind, könnte eine geeignete Teilmenge der Kacheln redundant dieselben Blöcke berechnen, anstatt jede Kachel ausschließlich disjunkte Blöcke zu verarbeiten – ein Kompromiss zwischen der in Abschnitt 9 optimierten Verifikationsgeschwindigkeit und einer höheren Fehlerinjektionsresistenz, dessen optimaler Parameterbereich (etwa $k = 12$ disjunkte plus 4 redundante Kacheln statt 16 rein disjunkter Kacheln) Gegenstand einer eigenen Optimierungsuntersuchung sein sollte.
- **Elektromagnetische Abstrahlanalyse (EMA):** Über die klassische DPA hinaus sind moderne Hardwaresicherheitsmodule zunehmend auch elektromagnetischen Seitenkanalangriffen ausgesetzt, bei denen lokale Abstrahlung einzelner Schaltungsteile – etwa einer einzelnen Sig-Engine-Kachel aus Abschnitt 9 – mit hochauflösenden Nahfeldsonden erfasst wird, wodurch räumlich lokalisierte Gegenmaßnahmen (wie die oben beschriebene Dummy-Last-Verteilung über mehrere Kacheln) lokal umgangen werden könnten. Eine vollständige EMA-Resistenzbewertung der k -fachen Kachelarchitektur, insbesondere die Frage, ob die räumliche Trennung der k Kacheln auf dem Die die EMA-Angriffsfläche eher vergrößert (mehr unterscheidbare Abstrahlquellen) oder verkleinert (geringere Einzellast je Quelle), ist eine offene, praktisch relevante

Frage für das physische Floorplanning in einer Folgearbeit.

10.4 Formale Verifikation der Zustandsmaschine

Während die mathematischen Eigenschaften der zugrunde liegenden Kryptoverfahren in Abschnitt 5 (sowie ergänzend für die parallele Architektur in Abschnitt 9) vollständig formal bewiesen wurden, ist die in Abschnitt 3 beschriebene Zustandsmaschine bislang nur informell spezifiziert. Für zukünftige Arbeiten wird eine Modellierung in einer Hardwarebeschreibungssprache mit formaler Verifikation (z. B. SystemVerilog Assertions oder ein Model Checker wie nuXmv) vorgeschlagen, um insbesondere folgende Sicherheitseigenschaften maschinell zu beweisen: (1) Aus jedem erreichbaren Zustand wird IDLE in endlich vielen Schritten wieder erreicht (Liveness, keine Deadlocks), (2) der Zustand WRITEBACK wird nur erreicht, wenn zuvor COMPUTE vollständig und ohne durch Fault-Injection gestörte Zwischenwerte durchlaufen wurde (Safety), und (3) das Kommandoregister HSM_CMD kann während eines laufenden COMPUTE-Zustands nicht erneut beschrieben werden (Atomarität gegen nebenläufige Kommandos von verschiedenen Applikationskernen).

Mit der in Abschnitt 9 eingeführten Parallelarchitektur erweitert sich der Katalog formal zu verifizierender Eigenschaften um mindestens drei weitere, nicht-triviale Aussagen, die über die ursprünglich für die sequenzielle Architektur formulierten Eigenschaften (1)–(3) hinausgehen: (4) *Vollständigkeit der Blockzuteilung*: Der Dispatcher aus Definition 6 muss garantiert jeden der N Blockindizes genau einer Kachel zuteilen (weder Lücken noch Duplikate in der Zuteilung – eine Eigenschaft, die formal als Bijektivität der Zuteilungsfunktion $i \mapsto (i \bmod k, \lfloor i/k \rfloor)$ nachgewiesen werden müsste); (5) *Synchronisation des Faltungsbaums*: Der XOR-Faltungsbaum darf erst dann mit der Kombination der Teilergebnisse beginnen, wenn *alle* k Kacheln den Zustand DONE erreicht haben (eine Barrieren-Synchronisationseigenschaft, deren Verletzung andernfalls zu einem falschen, auf veralteten Zwischenwerten basierenden Gesamtergebnis führen und damit Satz 8 in der *Hardware-Realisierung*, nicht im mathematischen Modell, verletzen würde); (6) *Fehlertoleranz bei Kachelausfall*: Sollte eine einzelne Kachel den Zustand FAULT statt DONE erreichen (etwa durch die in Abschnitt 10 unter Seitenkanalhärtung diskutierte Fehlerinjektion), muss die Gesamtzustandsmaschine dies zuverlässig erkennen und die gesamte Attestation als INVALID statt mit einem unvollständigen, potenziell falsch-positiven Ergebnis zurückmelden – eine Eigenschaft, die in der aktuellen, informellen Spezifikation aus Abschnitt 9 bereits intendiert, aber noch nicht maschinell nachgewiesen ist.

10.5 Weiterführende Optimierungen der Firmware-Attestierung für Radar-Steuergeräte

Der in Abschnitt 8 hergeleitete Wert von ≈ 7340 ms für die vollständige, sequenzielle Firmware-Attestierung eines 4 MB-Radar-ECU-Flash über HSM_SubgraphSig_Derive war das mit Abstand konkreteste quantitative Ergebnis dieser Arbeit. Die naheliegendste und wirksamste Gegenmaßnahme – eine massiv parallele Sig-Engine-Instanziierung – wurde in dieser Fassung der Arbeit nicht mehr nur als Ausblick skizziert, sondern in Abschnitt 9 vollständig zu einer formal als ergebnisinvariant bewiesenen Hardwarearchitektur ausgearbeitet: Eine 16-fache Sig-Engine-Kachelung reduziert die Verifikationszeit auf ≈ 459 ms und hält damit das 500-ms-Zeitbudget ein (Satz 9, Tabelle 5). Der verbleibende Ausblick konzentriert sich daher auf vier Weiterentwicklungsrichtungen, die auf der in Abschnitt 9 etablierten parallelen Grundarchitektur aufbauen und diese ergänzen, geordnet nach

erwarteter Wirksamkeit:

1. **Inkrementelle Attestierung statt vollständiger Neuberechnung.** Anstatt bei jedem Bootvorgang das gesamte 4 MB-Abbild neu zu signieren – selbst bei 16-facher Parallelisierung mit 459 ms Restlaufzeit gemäß Tabelle 5 –, wird vorgeschlagen, die Blocksignaturen $\sigma(A, j)$ einmalig nach jedem Firmware-Flash-Vorgang zu berechnen, in einem dedizierten, von Applikationssoftware nicht beschreibbaren HSM-Speicherbereich zu cachen, und beim eigentlichen Boot lediglich eine deutlich kleinere Stichprobe zufällig ausgewählter Blöcke (etwa nach dem in [1] beschriebenen Prinzip wiederholbarer, deterministischer Auswahl über einen Seed aus dem TRNG) neu zu berechnen und mit dem Cache abzugleichen. Diese Idee wirft jedoch eine offene mathematische Frage auf: Welche Stichprobengröße $m \ll 131\,072$ garantiert mit welcher Wahrscheinlichkeit die Erkennung einer böswilligen Manipulation einzelner Blöcke? Eine Anwendung der Chernoff-Schranke auf eine angenommene Gleichverteilung der Manipulationsposition liefert eine erste Abschätzung, dass bereits $m \approx 300$ zufällig gezogene Blöcke eine Erkennungswahrscheinlichkeit von über 99,9 % für eine Einzelblock-Manipulation ergeben würden, sofern der Angreifer die Stichprobenauswahl nicht vorhersagen kann – diese Abschätzung ist jedoch nicht Teil der in Abschnitt 5 bewiesenen Sätze und müsste in einer Folgearbeit formal hergeleitet und gegen adaptive Angreifer abgesichert werden, die gezielt versuchen, unentdeckte Bereiche zu manipulieren. In Kombination mit der k -fachen Kachelung aus Abschnitt 9 würde eine solche Stichprobenprüfung die Verifikationszeit von 459 ms auf $m/N \cdot 459 \text{ ms} \approx 1,0 \text{ ms}$ (für $m = 300$, $N = 131\,072$) reduzieren und damit erhebliche zusätzliche Zeitreserve für andere Boot-Aufgaben des Radar-ECU schaffen – eine Größenordnung, die selbst bei deutlich strengeren Zeitbudgets (etwa 50 ms) noch breite Sicherheitsmarge ließe.
2. **Hybridstrategie: schnelle Boot-Prüfung plus asynchrone Vollverifikation.** Auch bei eingehaltenem 500-ms-Budget durch die 16-fache Kachelung bleibt eine pragmatische Kombination sinnvoll: Beim eigentlichen Power-On wird ausschließlich `HSM.SignatureCipher.Verify` (Mikrosekunden-Latenz, siehe Abbildung 2) ausgeführt, um auch unter ungünstigen Bedingungen (geringere tatsächliche Taktfrequenz, zusätzliche Flash-Zugriffslatenz, vgl. die in Abschnitt 9 diskutierte fehlende Sicherheitsmarge bei $k = 16$) sicher innerhalb des Zeitbudgets zu bleiben, während die vollständige, parallelisierte `HSM.SubgraphSig.Derive`-Attestierung als asynchrone Hintergrundaufgabe nach Erreichen der Busfähigkeit durchgeführt wird. Wird dabei eine Manipulation festgestellt, löst das Radar-ECU einen sicheren Nachfehlerzustand (*Fail-Safe*, etwa Deaktivierung der ADAS-Funktion bei Beibehaltung der Grundfahrzeugfunktionen) aus. Diese Strategie verschiebt selbst die durch Abschnitt 9 bereits reduzierten 459 ms vollständig aus dem kritischen Boot-Pfad heraus, ohne auf die Sicherheitsgarantie aus Korollar 2 zu verzichten, und dient damit primär als zusätzliche Sicherheitsmarge gegenüber den in Abschnitt 9 genannten Worst-Case-Annahmen. In Kombination mit Punkt 1 dieses Abschnitts ergibt sich eine besonders robuste dreistufige Strategie: Mikrosekunden-Authentizitätsprüfung beim Power-On, Millisekunden-Stichprobenattestierung kurz danach, und vollständige Attestierung als seltene, asynchrone Hintergrundaufgabe (etwa einmal pro Fahrtzyklus statt bei jedem Boot).
3. **Erhöhung der Blockgröße n mit gleichzeitiger Pipeline-Tiefenoptimierung.** Da $c(n)$ gemäß Abschnitt 6 näherungsweise quadratisch in n wächst, während die

Anzahl der Blöcke quadratisch in n fällt, ist der Nettoeffekt einer Vergrößerung von n auf die Gesamtzeit T_{verify} in erster Näherung neutral – der eigentliche Hebel liegt daher nicht in n selbst, sondern in einer tieferen Pipeline-Parallelisierung *innerhalb* jeder einzelnen Sig-Engine-Kachel aus Abschnitt 9, etwa durch eine vollständig kombinatorische (statt sequenziell-akkumulierende) Berechnung der Zeilenkomponente $\rho(A, j)$ über einen Wallace-Baum-Addierer, was die pro Block benötigten Takte $c(n)$ von der aktuell angenommenen linearen Abhängigkeit von n auf eine logarithmische Abhängigkeit ($O(\log n)$ Gatterlaufzeiten) reduzieren könnte. Eine solche Optimierung würde *orthogonal* zur räumlichen Parallelisierung aus Abschnitt 9 wirken (mehr Kacheln *und* schnellere Einzelkacheln) und könnte in Kombination mit der bestehenden 16-fachen Kachelung die Verifikationszeit potenziell um einen weiteren Faktor 3 bis 5 reduzieren, abhängig vom konkreten Verhältnis der durch den Wallace-Baum eingesparten Gatterlaufzeiten zur weiterhin dominierenden Flash-Lesezugriffslatenz; eine belastbare Quantifizierung ist jedoch ohne konkrete Synthese-Ergebnisse einer Folgearbeit nicht seriös möglich und wird daher als eigenständiges VLSI-Entwurfsthema vorgeschlagen.

4. **Übertragbarkeit auf weitere automobile Steuergerätetypen.** Tabelle 3 (Abschnitt 8) zeigt, dass die hier diskutierte Problematik nicht auf Radar-ECUs beschränkt ist, sondern jedes Steuergerät mit Flash-Umfang oberhalb von etwa 0,3–0,5 MB bereits ohne Parallelisierung das typische Bootzeitbudget überschreitet. Domänen-Steuergeräte und zentrale Gateways mit 8 bzw. 16 MB Flash sind hiervon noch deutlich stärker betroffen (14,7 bzw. 29,4 Sekunden im sequenziellen Fall) und benötigen daher zwingend eine entsprechend höhere Kachelzahl k – nach Satz 9 etwa $k = 32$ für ein 8 MB-Domänensteuergerät bzw. $k = 64$ für ein 16 MB-Gateway, um dasselbe 500-ms-Budget einzuhalten –, kombiniert mit der Hybridstrategie aus Punkt 2 dieses Abschnitts, um die in Abschnitt 9 quantifizierten, mit k linear wachsenden Silizium- und Energiekosten (Abbildungen 11 und entsprechend skalierten Energiebedarf) für diese größeren Steuergerätetypen vertretbar zu halten. Für besonders kleine, ressourcenbeschränkte Steuergeräte (etwa 0,5 MB-Karosserie-Steuergeräte aus Tabelle 3) ist umgekehrt zu prüfen, ob eine geringere Kachelzahl ($k = 2$ oder $k = 4$) bereits ausreicht, um deren typischerweise ebenfalls knapperes Zeitbudget einzuhalten, ohne den in Abschnitt 9 quantifizierten Silizium-Mehraufwand unnötig zu treiben – eine produktlinienspezifische Kachelzahl-Konfiguration anstelle einer einheitlichen $k = 16$ -Vorgabe über alle Steuergerätetypen hinweg erscheint daher wirtschaftlich sinnvoll.

Zusammenfassend zeigt der Radar-ECU-Anwendungsfall exemplarisch, dass die in Abschnitt 5 bewiesene mathematische Korrektheit und Kollisionsfreiheit der Firmware-Attestierung eine notwendige, aber für den produktiven automobilen Einsatz keineswegs hinreichende Eigenschaft ist: Erst das Zusammenspiel aus formaler Sicherheit (Abschnitt 5), der nun in Abschnitt 9 etablierten parallelen Echtzeitarchitektur und den hier verbleibenden vier Weiterentwicklungsrichtungen macht den vorgestellten HSM-Funktionskern für sicherheitskritische, zeitbudgetierte Steuergeräte der Automobilindustrie vollständig einsatzreif.

10.6 Erweiterung auf Mehrkern-Parallelität

Die AURIX-TC3x-Familie verfügt über bis zu sechs Applikationskerne, die potenziell gleichzeitig HSM-Dienste anfordern. Die aktuelle Spezifikation in Abschnitt 3 sieht ein einzelnes, sequenziell abgearbeitetes Kommandoregister vor. Für zukünftige Versionen wird ein Anfrage-Scheduling mit Prioritätsstufen vorgeschlagen, bei dem sicherheitskritische Anfragen (`HSM_SecureBoot_Verify` während des Boot-Vorgangs) gegenüber Routineanfragen (`HSM_ModularCipher_*` für laufenden CAN-Verkehr) priorisiert werden, ohne dass dabei die in Satz 4 etablierte Versiegelungsgarantie des KeyStores durch nebenläufige Zugriffe unterlaufen werden kann – dies erfordert eine zusätzliche, noch zu spezifizierende Sperrlogik (Mutex) auf Registerebene.

Diese Mehrkern-Parallelität auf Applikationsebene (mehrere CPU-Kerne fordern unabhängig voneinander HSM-Dienste an) ist konzeptionell von der in Abschnitt 9 eingeführten Parallelität auf Funktionsebene (eine einzelne `HSM_SubgraphSig_Derive`-Anfrage wird intern auf k Kacheln verteilt) zu unterscheiden, beide Mechanismen sind jedoch nicht unabhängig voneinander zu betrachten: Während eine `HSM_SecureBoot_Verify`-Anfrage eines Applikationskerns die in Abschnitt 9 beschriebene k -fache Kachelung exklusiv für die Dauer der ≈ 459 ms-Berechnung belegt, könnten gleichzeitig eintreffende `HSM_ModularCipher_*`-Anfragen anderer Applikationskerne (etwa für laufenden CAN-Verkehr) entweder warten oder – bei entsprechender Erweiterung – eine kleine Teilmenge der Sig-Engine-Kacheln aus Abschnitt 9 zeitweise für die deutlich leichtgewichtigere Modular-ALU-Berechnung mitnutzen. Eine solche dynamische Ressourcenteilung zwischen Mehrkern-Scheduling und Funktionsparallelität ist architektonisch reizvoll, da sie die Silizium-Investition aus Abschnitt 9 auch außerhalb der relativ seltenen vollständigen Firmware-Attestierungen nutzbar macht, wirft jedoch erhebliche zusätzliche Komplexität bei der formalen Verifikation (vgl. vorigen Unterabschnitt, insbesondere Eigenschaft (5) zur Synchronisation) auf und wird daher als eigenständiges, anspruchsvolles Architekturthema für eine spätere Version (geschätzt Version 2.2 oder später) vorgeschlagen.

10.7 Standardisierung und Zertifizierung

Langfristig wird angestrebt, den hier vorgestellten HSM-Funktionskern einer Sicherheitszertifizierung nach Common Criteria (EAL4+) sowie einer funktionalen Sicherheitsbewertung nach ISO 26262 (ASIL-D-Tauglichkeit für sicherheitsrelevante Bootpfade) zu unterziehen. Hierfür ist insbesondere eine vollständige Dokumentation der in dieser Arbeit informell gebliebenen physikalischen Eigenschaften des TRNG (Entropiequalität nach NIST SP 800-90B, vgl. [10]) sowie eine unabhängige kryptoanalytische Begutachtung der Signatur-Chiffre und des Fenster-Chiffre-Protokolls durch Dritte erforderlich, da beide Verfahren – im Unterschied zu AES oder RSA – bislang nicht Gegenstand einer öffentlichen, jahrzehntelangen Kryptoanalyse-Historie sind. Die in dieser Arbeit erbrachten formalen Korrektheits- und Kollisionsfreiheitsbeweise (Abschnitt 5, ergänzt um die Beweise zur parallelen Architektur in Abschnitt 9) sind hierfür eine notwendige, aber keine hinreichende Voraussetzung; ergänzend wird eine systematische Suche nach strukturellen Schwächen (etwa algebraischen Angriffen, die die affine Struktur der Transformation $f(x) = (ax + b) \bmod p$ ausnutzen) als vorrangiger nächster Forschungsschritt empfohlen.

Ein realistischer Zertifizierungspfad würde sich grob in drei Phasen gliedern: Eine erste Phase (geschätzte Dauer ein bis zwei Jahre) umfasste die in den vorigen Unterabschnitten beschriebene formale Verifikation der Zustandsmaschine sowie eine erste, unabhängige kryptoanalytische Begutachtung der zugrunde liegenden Verfahren durch ex-

terne Gutachter; eine zweite Phase (ein weiteres Jahr) die Seitenkanal-Konformitätsprüfung nach etablierten Prüfschemata (etwa ISO/IEC 17825, dem internationalen Standard für Seitenkanal-Testmethoden); eine dritte Phase die eigentliche Common-Criteria- und ISO-26262-Begutachtung durch eine akkreditierte Prüfstelle. Dieser dreistufige Pfad ist mit dem üblichen Vorgehen etablierter automobiler HSM-Implementierungen (vgl. den nachfolgenden Vergleichsabschnitt) konsistent und sollte realistischerweise nicht vor Erreichen von Version 2.1 der in Abbildung 8 dargestellten Roadmap begonnen werden, da eine vorzeitige Zertifizierung einer noch nicht seitenkanalgehärteten Implementierung wirtschaftlich wenig sinnvoll wäre.

10.8 Vergleichende Einordnung gegenüber etablierten automobilen HSM-Standards

Der in dieser Arbeit vorgestellte Funktionskern unterscheidet sich grundlegend von etablierten automobilen Sicherheitsmodul-Standards wie SHE (*Secure Hardware Extension*, eine spezifizierte AUTOSAR-Schnittstelle für Sicherheitsfunktionen) oder dem Vorgängerkonzept EVITA, die beide ausschließlich auf standardisierten, jahrzehntelang kryptoanalytisierten Verfahren wie AES-128 beruhen. Eine ehrliche vergleichende Einordnung muss daher sowohl Stärken als auch Schwächen des hier vorgestellten Ansatzes benennen:

- **Vorteile gegenüber SHE/EVITA:** Die in Abschnitt 5 bewiesene exakte, nicht nur statistische Kollisionsfreiheit der Firmware-Attestierung (Satz 3) geht über das, was klassische AES-CMAC-basierte Integritätsprüfungen (wie sie SHE vorsieht) bieten, hinaus: Letztere besitzen eine zwar praktisch vernachlässigbare, aber theoretisch von Null verschiedene Kollisionswahrscheinlichkeit, während die in dieser Arbeit verwendete Signaturfunktion σ wegen ihrer Injektivität (Lemma 1) beweisbar kollisionsfrei ist. Die in Abschnitt 9 entwickelte parallele Architektur ist zudem in dieser Form – mit formal bewiesener Ergebnisinvarianz unabhängig vom Parallelisierungsgrad – über den hier vorgestellten Entwurf hinaus nicht Teil der bekannten SHE-Spezifikation.
- **Nachteile gegenüber SHE/EVITA:** Wie in Abschnitt 10 (Standardisierung) bereits angemerkt, fehlt der Signatur-Chiffre und dem Fenster-Chiffre-Protokoll die jahrzehntelange öffentliche Kryptoanalyse-Historie, die AES und RSA – und damit auch SHE – zugutekommt. Dies ist kein behebbarer technischer Mangel, sondern eine Frage der Zeit und der öffentlichen Begutachtung, die nur durch konsequente Offenlegung und Standardisierung (siehe vorigen Unterabschnitt) graduell geschlossen werden kann.
- **Mögliche Koexistenz statt Ersatz:** Aus den genannten Gründen erscheint es für eine erste Serieneinführung realistischer, den hier vorgestellten HSM-Funktionskern nicht als vollständigen Ersatz, sondern als *zusätzliche*, optionale Sicherheitsstufe neben einer SHE-konformen AES-Implementierung zu positionieren – vergleichbar mit dem in Abschnitt 4 bereits etablierten Defense-in-Depth-Prinzip von `HSM_SecureBoot_Verify`, das ja bereits heute einen unabhängigen AES-CMAC-Block als zweite Verifikationsstufe vorsieht. Eine solche Koexistenzstrategie würde es erlauben, von den in dieser Arbeit gezeigten Vorteilen (exakte Kollisionsfreiheit, parallele Echtzeitarchitektur) zu profitieren, ohne das etablierte Vertrauen in AES-basierte Verfahren vollständig aufzugeben, bis die in den vorigen Unterabschnitten beschriebene Standardisierung und unabhängige Kryptoanalyse abgeschlossen ist.

10.9 Wirtschaftliche und organisatorische Einordnung

Über die rein technische Betrachtung hinaus ist für eine reale Umsetzung des vorgestellten Entwurfs eine wirtschaftliche und organisatorische Einordnung notwendig, die in einer rein wissenschaftlichen Arbeit naturgemäß nur grob skizziert werden kann, hier aber aus Gründen der Vollständigkeit nicht ausgespart werden soll:

- **Silizium- und Entwicklungskosten:** Die in Abschnitt 9 geschätzten $\approx 3,0 \text{ mm}^2$ zusätzlicher Siliziumfläche für die 16-fache Sig-Engine-Kachelung sind im Kontext eines vollständigen Mikrocontroller-Dies moderat, jedoch nicht vernachlässigbar bei Massenfertigung im Bereich mehrerer Millionen Einheiten pro Jahr, wie sie für Radar-ECUs typisch ist; eine belastbare Kosten-Nutzen-Abwägung gegenüber lizenzierten, bereits qualifizierten AES/RSA-IP-Blöcken (wie sie SHE-konforme Implementierungen nutzen) wäre für eine Serienentscheidung unerlässlich, kann aber ohne konkrete Foundry- und Lizenzkostendaten in dieser Arbeit nicht seriös quantifiziert werden.
- **Entwicklungs- und Zertifizierungsaufwand:** Der im vorigen Unterabschnitt skizzierte, grob auf drei bis vier Jahre geschätzte Zertifizierungspfad (formale Verifikation, Seitenkanalprüfung, Common-Criteria/ISO-26262-Begutachtung) stellt einen erheblichen, mehrjährigen Aufwand dar, der bei einer Entscheidung für die Serieneinführung gegen den in Abschnitt 10.8 diskutierten Koexistenz-Ansatz (zusätzliche statt ersetzende Sicherheitsstufe) abzuwägen ist, da letzterer einen früheren, mit geringerem Zertifizierungsrisiko verbundenen Markteintritt ermöglichen würde.
- **Time-to-Market im Kontext der Post-Quanten-Migration:** Wie in der Diskussion der gitterbasierten Erweiterung dargelegt, besteht durch den Store-now-decrypt-later-Angriff ein nicht beliebig aufschiebbarer Migrationsdruck. Dies spricht dafür, die in dieser Arbeit vorgeschlagene Roadmap (Abbildung 8) nicht sequenziell, sondern mit überlappenden Entwicklungssträngen zu verfolgen – etwa die Seitenkanalhärtung (Version 2.1) und die gitterbasierte Migration (Version 1.2) parallel statt strikt nacheinander zu entwickeln –, auch wenn dies dem in Abschnitt 10.10 (Zusammenfassende Einordnung) beschriebenen Abhängigkeitsgraphen zuwiderläuft und entsprechend höheren Koordinationsaufwand zwischen den beteiligten Entwicklungsteams erfordert.
- **Open-Source- versus proprietäre Entwicklung:** Da die in [1, 2, 3, 4] zugrunde liegenden Verfahren öffentlich auf GitHub dokumentiert sind, böte eine offene, community-basierte Weiterentwicklung – ähnlich dem erfolgreichen Modell des NIST-Post-Quanten-Standardisierungsprozesses, aus dem etwa Kyber und Dilithium hervorgingen – eine Möglichkeit, die im vorigen Unterabschnitt als Nachteil benannte fehlende öffentliche Kryptoanalyse-Historie schneller zu schließen als durch rein interne, proprietäre Begutachtung; dies stünde jedoch im Spannungsfeld zu den üblichen Geheimhaltungsanforderungen der Automobilzulieferindustrie und müsste organisatorisch sorgfältig austariert werden.

10.10 Zusammenfassende Einordnung des Ausblicks

Die neun Ausblicksrichtungen dieser Arbeit – die in Abschnitt 9 nun bereits gelöste echtzeitfähige Parallelarchitektur, die hierauf aufbauenden weiterführenden Optimierungen

für Radar-Steuergeräte, gitterbasierte Post-Quanten-Migration, Firmware-Attestierung über Subgraph-Signaturen als Remote-Attestation-Protokoll, Seitenkanalhärtung, formale Verifikation, Mehrkern-Skalierung auf Applikationsebene, Standardisierung/Zertifizierung sowie die vergleichende und wirtschaftliche Einordnung – sind nicht unabhängig voneinander, sondern bauen in einer mehrschichtigen Abhängigkeitsstruktur aufeinander auf. Die in Abschnitt 9 formal bewiesene und in Abschnitt 10.5 weitergedachte parallele Echtzeitarchitektur ist die notwendige technische Grundlage dafür, dass die in Abschnitt 4.6 beschriebene Firmware-Attestierung überhaupt in zeitkritischen automobilen Boot-Pfaden produktiv genutzt werden kann. Eine belastbare Zertifizierung setzt ihrerseits sowohl diese nun etablierte Echtzeitfähigkeit als auch die Seitenkanalhärtung und die formale Verifikation der Zustandsmaschine (einschließlich der neu hinzugekommenen Dispatcher- und Faltungsbaum-Logik aus Abschnitt 9) voraus, während die Post-Quanten-Migration ihrerseits von einer hinreichend großen Flash- und Rechenkapazität abhängt, die erst durch die in Abschnitt 6 quantifizierte aktuelle Ressourcenbilanz als Ausgangspunkt bewertet werden kann. Die wirtschaftliche und vergleichende Einordnung schließlich rahmt diese rein technischen Abhängigkeiten in eine realistische Entwicklungs- und Markteinführungsperspektive ein, die – wie im vorigen Unterabschnitt diskutiert – für einzelne Richtungen (insbesondere die Post-Quanten-Migration) eine Abweichung von der streng sequenziellen, in Abbildung 8 dargestellten Versionsreihenfolge zugunsten überlappender Entwicklung nahelegen kann.

Die in Abbildung 8 dargestellte Reihenfolge der Versionen 1.0 bis 3.0 spiegelt die rein technische Abhängigkeitsstruktur wider, wobei die in Abschnitt 9 spezifizierte 16-fache Sig-Engine-Kachelung bereits für Version 1.0 als verbindlicher Bestandteil der Sig-Engine-Architektur vorgesehen werden sollte, um die in Abschnitt 8 aufgezeigte Lücke zwischen mathematischer Korrektheit und automobiler Echtzeitfähigkeit von Beginn an zu schließen, statt sie – wie in einer früheren Fassung dieser Arbeit – nachträglich als offene Frage zu behandeln. Insgesamt zeigt dieser ausführliche Ausblick, dass die in den Abschnitten 5 und 9 erbrachten formalen Beweise zwar das mathematische Fundament des vorgestellten HSM-Funktionskerns vollständig absichern, der Weg zu einer tatsächlichen, zertifizierten Serieneinführung in einem sicherheitskritischen automobilen Kontext jedoch noch erhebliche, hier im Detail benannte technische, organisatorische und wirtschaftliche Weiterentwicklungsschritte erfordert.

11 Zusammenfassung

Diese Arbeit hat gezeigt, wie die in den Arbeiten zur Signatur-Chiffre [2] und zum Fenster-Chiffre-Protokoll [3] entwickelten kryptographischen Verfahren konsequent auf einen konkreten Hardware-Funktionsentwurf für das HSM der Infineon-AURIX-TC3x-Familie übertragen werden können. Zehn HSM-Funktionen wurden spezifiziert, formal bewiesen korrekt rekonstruierend (Satz 2) und kollisionsfrei (Satz 3), sowie quantitativ hinsichtlich Taktzyklen, Energiebedarf und Speicherbedarf analysiert. Die Architektur folgt durchgehend einem Defense-in-Depth-Prinzip (unabhängige Zweitprüfung bei `HSM_SecureBoot_Verify`) und überträgt die in der Fenster-Chiffre etablierte Verschleierungs-idee konsequent auf die Schlüsselverwaltung.

Ergänzend wurde anhand des praxisrelevanten Anwendungsfalls eines Radarsteuergeräts mit 4 MB Programm-Flash formal hergeleitet (Satz 7) und numerisch ausgewertet, dass eine vollständige Firmware-Attestierung über `HSM_SubgraphSig_Derive` bei rein sequenzieller Spezifikation $\approx 7\,340$ ms beansprucht und damit das für sicherheitsrelevante automobiler Steuergeräte typische Boot-Zeitbudget von 100–500 ms deutlich überschreitet.

Diese Lücke wurde sodann vollständig geschlossen: Eine formal als ergebnisinvariant und kollisionsfrei bewiesene (Satz 8, Korollar 2) 16-fache parallele Sig-Engine-Kachelung reduziert die Verifikationszeit auf ≈ 459 ms (Satz 9, Tabelle 5) bei einem moderaten geschätzten Flächenmehrbedarf von $\approx 3,0$ mm² und hält damit das Radar-ECU-Zeitbudget erstmals zuverlässig ein, ohne dass dabei – wie formal bewiesen – irgendein Verlust an kryptographischer Sicherheit gegenüber der ursprünglichen sequenziellen Implementierung entsteht.

Der sehr ausführliche Ausblick (Abschnitt 10) ordnet diesen bereits gelösten Kern in neun zusammenhängende Weiterentwicklungsrichtungen ein. Auf der kryptographischen Ebene wurde die gitterbasierte Post-Quanten-Migration konkretisiert, einschließlich einer Architekturskizze der benötigten NTT-Pipeline, einer Diskussion des Speicherbudgets und einer Begründung dafür, warum der Migrationsdruck wegen des Store-now-decrypt-later-Risikos zeitlich nicht beliebig aufschiebbar ist. Auf der Protokollebene wurde die Firmware-Attestierung über Subgraph-Signaturen zu einem vollständigen Challenge-Response-Remote-Attestation-Protokoll ausgearbeitet, einschließlich eines konkreten Vorschlags für zählerbasierten Replay-Schutz ohne Echtzeituhr und eines aus der XOR-Algebra der parallelen Architektur (Satz 8) hergeleiteten Korollars zur inkrementellen Signaturaktualisierung bei Differential-Updates. Auf der physikalischen Sicherheitsebene wurden fünf konkrete Seitenkanal- und Fehlerinjektions-Härtungsmaßnahmen benannt, von der Maskierung der Modular-ALU bis zur elektromagnetischen Abstrahlanalyse der parallelen Kachelarchitektur. Auf der Verifikationsebene wurden sechs formal zu beweisende Eigenschaften der Zustandsmaschine identifiziert, davon drei neu durch die parallele Architektur hinzugekommen. Die verbleibenden Optimierungsrichtungen für das Radar-ECU – inkrementelle Stichprobenattestierung mit auf unter 1 ms geschätzter Restlaufzeit, eine dreistufige Hybridstrategie, Pipeline-Tiefenoptimierung und produktlinienspezifische Übertragbarkeit auf weitere Steuergerätetypen – wurden quantitativ unterlegt. Schließlich wurde der Entwurf einer vergleichenden Einordnung gegenüber dem etablierten automobilen SHE-Standard unterzogen (mit dem Ergebnis einer empfohlenen Koexistenzstrategie statt eines vollständigen Ersatzes) sowie einer wirtschaftlichen und organisatorischen Bewertung, die Silizium- und Zertifizierungskosten, Time-to-Market-Erwägungen im Kontext der Post-Quanten-Migration und die Frage einer offenen versus proprietären Weiterentwicklung diskutiert.

Insgesamt zeigt diese Arbeit damit nicht nur, dass der vorgestellte HSM-Funktionskern mathematisch korrekt, kollisionsfrei und – nach Einführung der parallelen Sig-Engine-Architektur – automobil-echtzeitfähig ist, sondern liefert mit dem ausführlichen Ausblick auch eine konkrete, in Abhängigkeiten strukturierte Landkarte der technischen, formalen, organisatorischen und wirtschaftlichen Schritte, die für eine tatsächliche, zertifizierte Serieneinführung in einem sicherheitskritischen automobilen Kontext noch zu gehen sind.

Literatur

- [1] Epp, S. (2026). *The Subgraph Algorithm*. <https://github.com/hjstephan86/subgraph>
- [2] Epp, S. (2026). *Die Signatur-Chiffre*. <https://github.com/hjstephan86/science>
- [3] Epp, S. (2026). *Modulares kryptografisches Verfahren mit Fenster-Protokoll (wchiffre)*. <https://github.com/hjstephan86/science>

- [4] Epp, S. (2026). *Gitterbasierte Kryptographie und Quanten-Fehlerkorrektur*. <https://github.com/hjstephan86/science>
- [5] Agrawal, M., Kayal, N., & Saxena, N. (2004). PRIMES is in P. *Annals of Mathematics*, 160(2), 781–793.
- [6] Buchmann, J. (2016). *Einführung in die Kryptographie* (6. Aufl.). Springer Spektrum.
- [7] Katz, J., & Lindell, Y. (2015). *Introduction to Modern Cryptography* (2. Aufl.). CRC Press.
- [8] Pomerance, C. (2005). Primality testing: variations on a theme of Lucas. *Congressus Numerantium*, 201–216.
- [9] Infineon Technologies AG (2023). *AURIX TC3xx Family User’s Manual – HSM Chapter*. Infineon Technologies AG, München.
- [10] National Institute of Standards and Technology (2018). *SP 800-90B: Recommendation for the Entropy Sources Used for Random Bit Generation*. NIST, Gaithersburg.
- [11] Regev, O. (2009). On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM*, 56(6), 1–40.
- [12] International Organization for Standardization (2018). *ISO 26262: Road vehicles – Functional safety*. ISO, Genf.